SOFTWARE ARCHITECTURES FOR ENTERPRISES · UNIVERSITÄT TRIER

ReflectAndMore

A field guide to Java meta-programming, or: how enterprise frameworks work, built by hand in eight stages

```
Candidate c = new Candidate(7);
// From here on, nothing about c is private any more.
```

Peter Sturm

Winter term 2026/27 · written with Claude

Contents

Introduction 4
- The small chill 4
- Who this is for 4
- Meet Candidate 5
- How the book and the code fit together 6
- The map 6
- A note on tone 7

Plain Java 8
- The program 8
- What the compiler decided 9
- Four things this program cannot do 9
- Where the names live 10

Run-time type information 12
- Where we stand 12
- Every object knows its class 12
- Class objects are ordinary values 13
- From a string to a type 14
- Walking the hierarchy 15
- Type tests and casts, without the syntax 15
- The compiler uses RTTI too 16
- Types that describe themselves 16
- Generics: erased from objects, kept in declarations 16
- Lambdas and anonymous classes have classes too 18

Reflection 20
- Where we stand 20
- Inspector: describing a class you have never seen 20
- Creating objects by name 23
- Calling methods by name 23
- Reading and writing fields, even final ones 24
- The wall 25
- Arrays 26
- Invoker: a convenience layer 26
- What does it cost? 27

Annotations 30
- Where we stand 30
- Defining an annotation and reading it back 30
- Meta-annotations: annotations all the way down 31
- Inspector lists annotations without knowing any 32
- Retention: who can see the annotation? 32
- Repeatable and inherited 33
- Annotations are interfaces; instances are proxies 34
- Framework one: declarative validation 35

Framework two: a test runner 35
Dynamic proxies 39
Where we stand 39
The pain, concretely 39
One handler for all interfaces 40
What the generated class looks like 41
A chain of handlers 42
Where do the annotations live? 44
A virtual proxy 44
A remote stub 45
Pitfalls 46
Dynamic class loading 48
Where we stand 48
Every class has a loader 48
Loading is not initializing 49
An implementation chosen by configuration 50
ServiceLoader: the JDK's own plugin registry 50
A plugin compiled while the program runs 51
Two versions, side by side 52
Consequences 53
Inversion of control 55
Where we stand 55
Dependency injection by hand 55
The alternative, and why it hides things 56
The container finds its components 57
One request builds the graph 57
Qualifiers 59
Scopes 59
Field injection reaches into Candidate 60
Cycles 60
The other face of inversion of control 60
Where control went 61
Generating classes at run time 63
Where we stand 63
Reading bytecode 63
Writing a class from nothing 64
A proxy for a class 65
Hidden classes 66
Looking at the result with javap 67
The whole spectrum 67
Conclusion 69
What we built 69
The one idea 69
What it costs, in one place 70
What to do next 70

Introduction

In which we promise to take the magic out of enterprise Java, and introduce the most boring class ever written.

The small chill

Every Java developer has had the moment. You write a class, you put `@Component` on top of it, you declare a field with `@Inject` or `@Autowired`, and you never write `new`. The object simply *is there* when you need it, with all its collaborators attached. You add `@Transactional` to a method and a database transaction wraps itself around every call. You add `@Test` and a method you never call is called for you. You return an object from a REST endpoint and it leaves the building as JSON.

It works, and it is wonderful, and somewhere in the back of your mind there is a small chill: *I have no idea how any of this happens.*

This book is about that chill. In eight stages it builds, from nothing but the JDK, every mechanism that makes a modern application framework tick: run-time type information, reflection, annotations, dynamic proxies, dynamic class loading, a dependency-injection container, and finally classes written as raw bytecode while the program runs. At the end you will have a tiny Spring, a tiny JUnit, a tiny Hibernate proxy and a tiny RMI in a few hundred lines, and you will know exactly why they work and what they cost. Frameworks are not magic. They are reflection, annotations, proxies and class loaders, arranged with taste.

Who this is for

You can program in Java. You know what an interface is, you have written a class with a constructor, and you have used a `List<String>`. You have probably used a framework and felt the chill. You have never, or hardly ever, called `getClass()` on purpose, and you have never defined your own annotation. That is the audience: people who are good at the language and have not yet looked under its floorboards.

Everything here is plain JDK 25. There are no third-party libraries, because a library would be another place for magic to hide. If a program in this book does something, you can open the source and see the whole mechanism.

Meet Candidate

Textbooks love to explain reflection with a `Customer` class that has an `Order` with `LineItems`. We will not do that. A business domain is a distraction: half of your attention goes to wondering whether the discount logic is realistic, and the mechanism being explained gets the other half. So our protagonist is deliberately, proudly boring. It is called `Candidate`, because it is the candidate for whatever we do next: inspection, interrogation, wiretapping, cloning.

```
public class Candidate {

    private static int created;        // counts instances

    private int x;
    private final String label;

    public Candidate(int x) {
        this.x = x;
        this.label = "candidate#" + x;
        created++;
    }

    public Candidate() {
        this(0);
        reset();
    }

    private Candidate(String label) {  // only reachable from inside the class
        this.x = -1;
        this.label = label;
        created++;
    }

    public int getX() { return x; }

    public void f(int y, double d) {
        if (d < 0) x = y; else x = -y;
    }

    protected void reset() { x = 42; }

    private String secret() { return "the secret of " + label; }

    public static int created() { return created; }

    @Override
    public String toString() {
        return "Candidate{x=" + x + ", label='" + label + "'}";
    }
}
```

stage00-plain/src/main/java/sa4e/reflect/Candidate.java

That is all there is. Three constructors of three visibilities, a private field, a private final field, a static counter, a public method with two parameters, a protected method, a private method, a static method. There is nothing to understand about what `Candidate` *does*, which leaves all your attention for what we do *to* it. Everything else in the book is named after its role in the mechanism: `Inspector`, `Invoker`, `Validator`, `Interceptor`, `Container`, `Plugin`, `Dependency`. When you later meet Spring's `BeanFactory` or Hibernate's `LazyInitializer`, you will recognise the roles.

`Candidate` changes as little as possible from stage to stage. It gains an annotation here, an interface there, an injected field near the end, and each change is announced. Everything that grows is the framework around it.

How the book and the code fit together

The code lives in a repository with one folder per stage:

```
ReflectAndMore/
  stage00-plain/        stage04-proxies/
  stage01-rtti/         stage05-loading/
  stage02-reflection/   stage06-ioc/
  stage03-annotations/  stage07-codegen/
```

Each folder is a complete, runnable program. All of them use the same package `sa4e.reflect` and the same entry point `sa4e.reflect.Main`, so the difference between two neighbouring folders is exactly one lecture step; `tools/diff-stages.sh 03` shows what stage 03 added to stage 02. Run a stage with nothing but a JDK:

```
./run.sh 03
```

Every program prints Markdown. In the terminal that reads fine; redirected into a file it becomes a document with headings and tables, and each stage folder contains such a file, `output.md`, from the last run. The output excerpts in this book are taken from those files, so what you read here is what the program really printed.

Every chapter follows the same rhythm, because every stage does:

1. **Where we stand.** A concrete limitation of the previous stage. Not a philosophical one; a line of code that cannot be written.
2. **The mechanism.** The JDK API that removes the limitation, in the smallest form that works, walking through the sections of the stage's `Main`.
3. **In the wild.** Which framework does this for a living, and how far our toy is from the real thing.
4. **The bill.** What it costs: performance, security, debuggability, build-time tooling.
5. **Try it yourself.** Exercises. The best way to believe that a container is 150 lines is to add a feature to it.

The map

Stage	Question it answers	Mechanism
00 plain	What does the program look like without any meta-programming?	Nothing yet. Compile-time binding.
01 rtti	What does the JVM know about `Candidate` at run time?	`Class` objects, hierarchy, `Class.forName`, sealed types, records, erasure

02 reflection	Can a program read and operate `Candidate` without knowing it at compile time?	`Inspector`, `Invoker`, private access, the encapsulation wall, `MethodHandle`
03 annotations	How does `Candidate` tell a framework something extra?	`@Comment`, retention, repeatable and inherited annotations, a validator, a test runner
04 proxies	How do we add behaviour around `Candidate` without touching it?	`java.lang.reflect.Proxy`, handler chains, transactions, lazy loading, remote stubs
05 loading	How does code arrive that did not exist at compile time?	Class loaders, configuration, `ServiceLoader`, run-time compilation, class identity
06 ioc	Who creates `Candidate` and fills its dependencies?	A container with scanning, injection, scopes, cycles, an event bus
07 codegen	What if the class itself is written at run time?	ClassFile API, generated subclasses, hidden classes

The stages are cumulative in understanding, not always in code. A helper class travels forward only while a later stage still uses it, so each folder stays focused on its own idea.

A note on tone

We will be unkind to `Candidate`'s privacy. We will read its private fields, call its private methods, change a field it declared `final`, and in the last chapter we will write a subclass of it in raw bytecode without asking. None of this is bad practice in itself; it is *exactly* what Jackson, JUnit, Spring and Hibernate do to your classes every day. The point of doing it by hand once is that you will afterwards know when a framework is doing it to you, and what it means when the JVM says no.

Let us begin where every program begins: with nothing happening at all.

STAGE 00

Plain Java

In which nothing happens, and that is the point.

The program

Stage 00 uses `Candidate` the way every Java program uses every class. It creates two instances, calls a method, prints the results. There is no trick, which makes it the baseline against which every later trick is measured.

```
public final class Main {

    public static void main(String[] args) {
        Report r = Report.stdout();
        r.title("Stage 00: plain Java");

        Candidate a = new Candidate(7);
        Candidate b = new Candidate();
        a.f(3, -1.0);

        r.heading("Using Candidate the ordinary way");
        r.kv("a = new Candidate(7); a.f(3, -1.0)", a);
        r.kv("b = new Candidate()", b);
        r.kv("Candidate.created()", Candidate.created());

        r.heading("What this program cannot do");
        r.item("describe Candidate (fields, constructors, methods) without its source");
        r.item("call secret() or the private constructor");
        r.item("create an instance of a class whose name is known only at run time");
        r.item("decorate, replace or wire Candidate without editing Main");
        r.flush();
    }
}
```

stage00-plain/src/main/java/sa4e/reflect/Main.java

`Report` is a small helper that prints Markdown: a title, headings, `kv` for an expression and its result, `item` for bullet points. It is identical in every stage and has nothing to do with the material; think of it as `System.out.println` in a nicer suit. The output:

Expression	Result
`a = new Candidate(7); a.f(3, -1.0)`	`Candidate{x=3, label='candidate#7'}`
`b = new Candidate()`	`Candidate{x=42, label='candidate#0'}`
`Candidate.created()`	`2`

Two candidates, one with `x` set by `f` because `d` was negative, one reset to 42 by the no-arg constructor. Nothing you could not have predicted by reading the source. That is the whole point of stage 00: *everything* about this program was decided by reading source.

What the compiler decided

It is worth being precise about what "decided at compile time" means, because the rest of the book is about moving decisions from that moment to a later one.

When `javac` compiles `a.f(3, -1.0)`, it has `Candidate.java` in front of it. It looks up `f`, checks that there is a method of that name taking an `int` and a `double`, checks that `f` is visible from `Main` (it is public), and emits an instruction. Run `javap -c` on the compiled `Main` and you see it:

```
28: aload_2
29: iconst_3
30: ldc2_w        #25     // double -1.0d
33: invokevirtual #27     // Method sa4e/reflect/Candidate.f:(ID)V
```

Three things in these four lines matter for everything that follows.

First, the *check* happened at compile time. If `f` were private, `javac` would have refused, and no class file would exist. Visibility is, first of all, a compiler concept.

Second, the *binding* to the name `f` is still symbolic. The instruction does not say "jump to address 0x4a20"; it says "call the method `f` with descriptor `(ID)V` on class `sa4e/reflect/Candidate`". The class file keeps names. The JVM resolves that name when the instruction first runs, by looking into the loaded `Candidate` class. Keep this in mind: the names survive compilation, which is the reason reflection is possible at all. A C++ compiler would have thrown them away.

Third, the name is a *literal* in `Main`'s constant pool. `Main` can only ever call methods whose names were typed into `Main.java`. It cannot call a method whose name arrives in a configuration file, and it cannot create an instance of a class whose name arrives over the network.

Four things this program cannot do

The second half of `Main` lists them, and each one is a job that some framework does for a living.

Describe Candidate without reading its source. A debugger, a JSON serializer, an object-relational mapper and a test runner all need to know what fields and methods a class has, for a class they have never seen. In stage 00 the only description of `Candidate` is the hand-written `toString()`, which is wrong the moment someone adds a field and forgets to update it. Stages 01 and 02 fix this.

Call `secret()` or the private constructor. The compiler says no, and that is usually right. But a serializer must set private fields when it reads JSON, a test may need to construct an object through a constructor the author hid, and a dependency-injection container needs to fill a private field without a setter. Stage 02 shows how the door opens, and where the JVM has put a wall.

Create an instance of a class named at run time. `new Candidate(7)` requires the word `Candidate` in the source. A plugin system, a JDBC driver manager, a configuration file saying

`implementation=com.acme.FastSorter`, all need to go from a string to an object. Stages 01 and 05 are about that door.

Decorate, replace or wire Candidate without editing Main. To log every call to `f`, Main would have to call a logging wrapper instead. To run `f` inside a transaction, Main would have to begin and commit it. To give `Candidate` a collaborator, Main must create both and hand one to the other. In stage 00 Main does all the wiring by hand, and in a real program Main becomes a thousand lines of `new`. Stages 04 and 06 take that job away from Main.

> **Note**
>
> Notice that none of the four is about `Candidate` being badly designed. `Candidate` is fine. The limitations are limitations of *compile-time binding*: every decision is made by `javac` from source text, and `javac` is long gone when the program runs. Meta-programming is the art of making some of those decisions later, with information that only exists later.

Where the names live

One experiment is worth doing right now, because it foreshadows everything. Compile stage 00 and run the JDK's disassembler on `Candidate` itself:

```
./run.sh 00 > /dev/null          # compiles into stage00-plain/out/classes
javap -p -cp stage00-plain/out/classes sa4e.reflect.Candidate
```

```
public class sa4e.reflect.Candidate {
  private static int created;
  private int x;
  private final java.lang.String label;
  public sa4e.reflect.Candidate(int);
  public sa4e.reflect.Candidate();
  private sa4e.reflect.Candidate(java.lang.String);
  public int getX();
  public void f(int, double);
  protected void reset();
  private java.lang.String secret();
  public static int created();
  public java.lang.String toString();
}
```

Every name, every type, every modifier is in the class file, private ones included. `javap` is not doing anything clever; it reads a well-documented binary format. If a command-line tool can read this, so can a running program, and in the next two stages it will.

> **Takeaways**
>
> - In plain Java, every decision about which class, which method and which field is made by the compiler from source text.
> - The compiled class file still contains all names, types and modifiers. Compilation hides nothing; it only fixes the decisions.

> - The four things stage 00 cannot do, describing, reaching private members, creating by name, and wiring without editing `Main`, are exactly the jobs of serializers, test runners, plugin systems and containers.

STAGE 01

Run-time type information

In which every object turns out to know exactly what it is, and `Class` turns out to be a thing you can hold in your hand.

Where we stand

Stage 00 could use `Candidate` but could not say anything *about* it. Given a variable of type `Candidate`, Main could not find out whether the object was really a `Candidate` or a subclass; it could not compare two types; it could not take the string `"sa4e.reflect.Candidate"` and turn it into an object. All of that is compile-time knowledge, and at run time it was gone.

Except that it is not gone. This chapter is about the fact that the JVM keeps a complete description of every type, that this description is itself an object, and that you can ask it questions. Nothing is read or changed yet; that is stage 02. Stage 01 only looks.

Every object knows its class

Here is the fact that makes everything else possible. Every object on the JVM heap starts with a header, and the header contains a pointer to the object's class. Not a type tag, not an integer, a pointer to a full description: name, superclass, interfaces, fields, methods, constant pool, and a reference to a `java.lang.Class` object that represents the class at the Java level. `getClass()` follows that pointer. It is one of the cheapest calls in the language, and it never lies.

Stage 01 adds a subclass so that there is something to lie about:

```
public class Special extends Candidate implements Comparable<Special> {
    public Special(int x) { super(x); }

    @Override
    public int compareTo(Special other) {
        return Integer.compare(getX(), other.getX());
    }
}
```

and asks the obvious question:

```
Candidate c = new Special(7);        // static type Candidate, dynamic type Special
r.kv("c.getClass()", c.getClass());
```

```
r.kv("c.getClass().getName()", c.getClass().getName());
r.kv("c instanceof Special", c instanceof Special);
```

Expression	Result
`Candidate c = new Special(7)`	`declared type Candidate`
`c.getClass()`	`class sa4e.reflect.Special`
`c.getClass().getName()`	`sa4e.reflect.Special`
`c instanceof Special`	`true`

The variable `c` has the *static* type `Candidate`: that is what the compiler knows and what it will let you call. The object has the *dynamic* type `Special`: that is what the JVM knows. `instanceof` and virtual method dispatch have always used the dynamic type; `getClass()` simply hands it to you.

Class objects are ordinary values

`c.getClass()` returns a `Class<?>`, and `Class` is a perfectly ordinary Java class. Its instances can be stored in variables, put in maps, passed to methods, compared with `==`. There is a literal syntax for them, `Candidate.class`, and there is one for every type in the language, including the ones that are not classes at all:

```
Class<?>[] samples = {int.class, Integer.class, int[].class, String[][].class,
        Candidate.class, Special.class, Variant.class, Variant.A.class,
        Variant.Mode.class, Runnable.class, void.class};
```

getName()	getSimpleName()	getCanonicalName()	kind
`int`	int	int	primitive
`java.lang.Integer`	Integer	java.lang.Integer	class
`[I`	int[]	int[]	array of int
`[[Ljava.lang.String;`	String[][]	java.lang.String[][]	array of String[]
`sa4e.reflect.Candidate`	Candidate	sa4e.reflect.Candidate	class
`sa4e.reflect.Special`	Special	sa4e.reflect.Special	class
`sa4e.reflect.Variant`	Variant	sa4e.reflect.Variant	sealed interface
`sa4e.reflect.Variant$A`	A	sa4e.reflect.Variant.A	record
`sa4e.reflect.Variant$Mode`	Mode	sa4e.reflect.Variant.Mode	enum
`java.lang.Runnable`	Runnable	java.lang.Runnable	interface
`void`	void	void	primitive

A few things in this table surprise people the first time.

`int.class` exists and is not `Integer.class`. Primitives have Class objects so that reflection can describe a method like `f(int, double)` precisely; a parameter of type `int` and a parameter of type `Integer` are different things, and the JVM must know which one to box.

Arrays are classes, with names that look like line noise. `[I` is "array of int", `[[Ljava.lang.String;` is "array of array of String". These are the JVM's internal descriptors leaking through. `getSimpleName()` gives the human form, `getName()` gives the form that `Class.forName()` understands.

Nested classes use `$` in `getName()` and `.` in `getCanonicalName()`. `Variant$A` is the real name as far as the JVM and the class file are concerned; `Variant.A` is the Java-language spelling. Frameworks that take class names from configuration files must accept the `$` form, and the error messages of frameworks that do not are a small genre of their own.

Even `void` has a Class object, so that `getReturnType()` can return *something* for a `void` method.

There is exactly one `Class` object per loaded class, which is why `==` works:

Expression	Result
`Candidate.class == c.getClass()`	`false`
`Candidate.class == new Candidate().getClass()`	`true`

The first line is `false` because `c` is a `Special`. A `Class` object identifies a type exactly; it does not know about "is a kind of". For that there is `isAssignableFrom`, further down.

From a string to a type

Now the door that everything in the second half of the book walks through:

```
Class<?> byName = Class.forName("sa4e.reflect.Candidate");
Object instance = byName.getDeclaredConstructor().newInstance();
```

Expression	Result
`Class.forName("sa4e.reflect.Candidate")`	`class sa4e.reflect.Candidate`
`byName.getDeclaredConstructor().newInstance()`	`Candidate{x=42, label='candidate#0'}`
`Class.forName("Candidate")`	`ClassNotFoundException (the package is part of the name)`

Read the first two lines slowly. The word `Candidate` does not appear as a type anywhere in them. A *string* went in, an *object* came out, and the string could have come from a properties file, a command-line argument, a database row or a network packet. This is how a JDBC driver manager finds `org.postgresql.Driver`, how a logging facade finds its back-end, how a plugin system finds plugins, and how a dependency-injection container finds everything. We are getting ahead of ourselves, and `getDeclaredConstructor().newInstance()` is really stage 02 material, but it was impossible to resist showing the door open.

The third line is the first error of the book and a common one: `Class.forName` wants the fully qualified name. There is no "current package" at run time; packages are part of the name, full stop.

> **Note**
>
> `Class.forName(name)` does three things: it *loads* the class if it is not loaded yet, it *links* it, and it *initializes* it, which means running its static initializers. The third step is a side effect, and

> it is deliberate: for many years JDBC drivers registered themselves in a static initializer, and `Class.forName("org.postgresql.Driver")` was the idiom to trigger it. Stage 05 takes this apart.

Walking the hierarchy

A `Class` object knows where it sits:

Expression	Result
`superclass chain of Special`	`Special -> Candidate -> Object`
`Special.class.getInterfaces()`	`[interface java.lang.Comparable]`
`all interfaces of ArrayList (transitively)`	`[List, SequencedCollection, Collection, Iterable, RandomAccess, Cloneable, Serializable]`
`Object.class.getSuperclass()`	`null`

`getSuperclass()` returns one class or `null` at the top; `getInterfaces()` returns only the interfaces this class declares directly, so "all interfaces of ArrayList" needs a little recursion over superclasses and super-interfaces. The loop that produced the first line is three lines of code:

```
for (Class<?> k = Special.class; k != null; k = k.getSuperclass()) {
    chain.append(k.getSimpleName()).append(k.getSuperclass() != null ? " -> " : "");
}
```

Every serializer walks exactly this loop to find the fields of superclasses, and every container walks it to find annotations on them.

Type tests and casts, without the syntax

Everything `instanceof` and the cast operator do is also available as a method call on a `Class` object, which matters when the class is a variable rather than a name in the source:

Expression	Result
`Candidate.class.isAssignableFrom(Special.class)`	`true`
`Special.class.isAssignableFrom(Candidate.class)`	`false`
`Special.class.isInstance(c)`	`true`
`Special.class.cast(c).getX()`	`7`
`Variant.A.class.cast(c)`	`ClassCastException`

`a.isAssignableFrom(b)` reads as "a variable of type `a` can hold a `b`". The direction trips everyone up at least once; the mnemonic is that it mirrors an assignment `a = b`. `isInstance` is `instanceof` with the type on the left, and `cast` is the cast operator, including the `ClassCastException` when it fails. The one new thing `cast` gives you is a *typed* return value: `Special.class.cast(c)` is a `Special` as far as the compiler is concerned, so you can call `getX()` on it directly.

When stage 06 asks "which registered classes implement `Capability`?", the answer is one `isAssignableFrom` per registered class. That is the whole algorithm.

The compiler uses RTTI too

Modern Java has pattern matching, and it is worth pausing to notice that it is built on the same foundation:

```
public sealed interface Variant permits Variant.A, Variant.B {
    record A(int value, String tag) implements Variant {}
    final class B implements Variant { /* ... */ }
    enum Mode { ON, OFF, UNKNOWN }
}
```

```
String text = switch (v) {                  // no default branch needed
    case Variant.A a -> "A with value " + a.value() + " and tag " + a.tag();
    case Variant.B b -> "B with weight " + b.weight();
};
```

Expression	Result
`switch (v) for a A`	`A with value 1 and tag one`
`switch (v) for a B`	`B with weight 2.5`

The switch compiles to type tests against the object's class, the same test `isInstance` performs. And because `Variant` is *sealed*, the compiler knows that `A` and `B` are the only possibilities and does not demand a `default`. Sealed types are RTTI that the compiler can reason about; the next section shows that the run time can see it as well.

Types that describe themselves

Sealed interfaces, records and enums all carry extra structure, and all of it is visible at run time:

Expression	Result
`Variant.class.isSealed()`	`true`
`Variant.class.getPermittedSubclasses()`	`[A, B]`
`Variant.A.class.isRecord()`	`true`
`Variant.A.class.getRecordComponents()`	`value: int, tag: String`
`Variant.B.class.isRecord()`	`false`
`Variant.Mode.class.isEnum()`	`true`
`Variant.Mode.class.getEnumConstants()`	`[ON, OFF, UNKNOWN]`

`getRecordComponents()` deserves a highlight. A record declares its components once, in the header, and the JVM remembers them in order, with names and types. A serializer that meets a record does not have to guess which fields matter or which constructor to call: the components are the fields, the canonical constructor takes them in exactly this order, and the accessors have exactly these names. Records were designed, among other things, to make reflection boring, and stage 02 will make use of that.

Generics: erased from objects, kept in declarations

Here is the one place where the "every object knows its class" story has a hole:

Expression	Result
`new ArrayList<String>().getClass() == new ArrayList<Integer>().getClass()`	`true`

The two lists have the same class. Type arguments are erased when the compiler is done with them; at run time there is only `ArrayList`. This is the famous *type erasure*, and it is the reason you cannot write `new T()` or `list instanceof List<String>`.

But erasure applies to *objects*, not to *declarations*. A field declared as `List<String>`, a method returning `Map<String, List<Integer>>`, a class extending `TypeRef<List<String>>`: these declarations keep their full generic signature in the class file, and reflection can read it back:

```
static class Holder {
    List<String> items = new ArrayList<>();
    Map<String, List<Integer>> index = Map.of();
}
```

Expression	Result
`field items: getType()`	`List`
`field items: getGenericType()`	`java.util.List<java.lang.String>`
`field index: getType()`	`Map`
`field index: getGenericType()`	`java.util.Map<java.lang.String, java.util.List<java.lang.Integer>>`

`getType()` is the erased class; `getGenericType()` is the declaration, as a `java.lang.reflect.Type` that can be a `ParameterizedType` with actual type arguments. This is how Jackson knows that the field `items` should be filled with strings, not with `Object`s, when it reads JSON.

And then there is the trick that every Java developer should see once, because it looks like black magic and is not:

```
public abstract class TypeRef<T> {
    private final Type type;

    protected TypeRef() {
        Type superclass = getClass().getGenericSuperclass();
        if (!(superclass instanceof ParameterizedType parameterized)) {
            throw new IllegalStateException("use: new TypeRef<SomeType>() {}");
        }
        this.type = parameterized.getActualTypeArguments()[0];
    }

    public Type type() { return type; }
}
```

```
TypeRef<List<String>> ref = new TypeRef<>() {};
r.kv("new TypeRef<List<String>>() {}.type()", ref.type());
```

Expression	Result
`new TypeRef<List<String>>() {}.type()`	`java.util.List<java.lang.String>`

The `{}` at the end is the whole trick. It creates an *anonymous subclass* of `TypeRef<List<String>>`, and that subclass is a declaration, so its generic superclass survives erasure. The constructor reads

it back. Jackson's `TypeReference`, Gson's `TypeToken` and Guice's `TypeLiteral` are this class, and now you know why you have to write those curly braces.

Lambdas and anonymous classes have classes too

The last section is a teaser for stage 07:

Expression	Result
`lambda.getClass().getName()`	`sa4e.reflect.Main$$Lambda/0x0000018001046660`
`lambda.getClass().isHidden()`	`true`
`anonymous.getClass().getName()`	`sa4e.reflect.Main$2`
`anonymous.getClass().isAnonymousClass()`	`true`

An anonymous class is compiled by `javac` into an ordinary class file called `Main$2.class`. A lambda is not: there is no class file for it anywhere. The class `Main$$Lambda/0x...` is generated by the JVM the first time the lambda expression runs, and it is *hidden*: it has no findable name, `Class.forName` cannot see it, and it disappears when it is no longer referenced. Stage 07 generates classes exactly like this, so remember the word "hidden".

In the wild

- Every DI container starts with `isAssignableFrom` to find candidates for an interface, and with `Class.forName` to turn configuration into classes.
- JDBC (`Class.forName("org.postgresql.Driver")`), JNDI, logging back-ends, JAX-RS runtimes and plugin systems all begin with a class name in a string.
- Jackson's `TypeReference<List<Customer>>`, Gson's `TypeToken` and Guice's `TypeLiteral` are `TypeRef`.
- Hibernate reads `getGenericType()` of every collection-typed field to know what an `@OneToMany` contains.

The bill

Almost nothing. `getClass()` is a pointer dereference, `isInstance` is the same check as `instanceof`, and the `Class` objects already exist because the JVM needs them anyway. The one expensive and side-effect-laden call is `Class.forName`, which may load and initialize a class; stage 05 is about exactly that.

Try it yourself

1. Print the full inheritance chain of `java.util.LinkedHashMap` including all interfaces, transitively, without duplicates.
2. Write `boolean sameHierarchy(Object a, Object b)` that returns true when one object's class is a subclass of the other's, without using `instanceof`.
3. Make `TypeRef` work for nested arguments such as `Map<String, List<Integer>>` and print the inner types recursively. (Hint: `ParameterizedType` is itself a `Type`.)

Takeaways

- Every object knows its class; `getClass()` returns the dynamic type, the one the JVM uses for dispatch.
- `Class` objects are values: one per loaded type, comparable with `==`, obtainable from a literal, from an object or from a string.
- `isAssignableFrom`, `isInstance` and `cast` are `instanceof` and the cast operator for cases where the type is a variable.
- Sealed types, records and enums describe their own structure, which makes them easy for frameworks.
- Generics are erased from objects but kept in declarations; the super type token exploits exactly that.

STAGE 02

Reflection

In which we read `Candidate` like a book, call what we are not supposed to call, and discover that the JDK has built itself a wall.

Where we stand

Stage 01 could ask a class *what* it is. It could not ask what the class *contains*, and it could not do anything with the answer: no object creation with arguments, no method call by name, no access to a field unless the field's name was typed into the source. The `Inspector` that this chapter builds answers the first question for any class in the world, and the second half of the chapter does the rest: create, call, read, write, by name, at run time.

The package is `java.lang.reflect`, and its central idea is that every *member* of a class is itself an object. A `Constructor`, a `Field`, a `Method`, a `Parameter`: each can be listed, inspected and used. The `Class` object from stage 01 is the index; these are the entries.

Inspector: describing a class you have never seen

The lecture slides call this example "TalkAbout". It takes a `Class<?>` and produces an XML description of it, and it does so without instantiating the class, without importing it, and without knowing it at compile time:

```
public final class Inspector {
    private final Class<?> target;

    public Inspector(Class<?> target) { this.target = target; }

    public String describe() {
        StringBuilder xml = new StringBuilder();
        xml.append("<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n");
        xml.append("<class name=\"").append(target.getName())
           .append("\" modifiers=\"").append(Modifier.toString(target.getModifiers()))
           .append("\" superclass=\"").append(target.getSuperclass().getName())
           .append("\">\n");

        // The reflection API makes no promise about the order of members, so sort.
        Constructor<?>[] constructors = target.getDeclaredConstructors();
        Arrays.sort(constructors, Comparator.comparing(Constructor::toGenericString));
        for (Constructor<?> ctor : constructors) {
```

```
            xml.append("  <constructor modifiers=\"")
               .append(Modifier.toString(ctor.getModifiers())).append("\">\n");
            appendParameters(xml, ctor);
            xml.append("  </constructor>\n");
        }

        Field[] fields = target.getDeclaredFields();
        Arrays.sort(fields, Comparator.comparing(Field::getName));
        for (Field f : fields) {
            xml.append("  <field name=\"").append(f.getName())
               .append("\" type=\"").append(f.getType().getName())
               .append("\" modifiers=\"").append(Modifier.toString(f.getModifiers()))
               .append("\"/>\n");
        }

        Method[] methods = target.getDeclaredMethods();
        Arrays.sort(methods, Comparator.comparing(Method::getName));
        for (Method m : methods) {
            if (m.isSynthetic()) continue;        // compiler-generated helpers
            xml.append("  <method name=\"").append(m.getName())
               .append("\" modifiers=\"").append(Modifier.toString(m.getModifiers()))
               .append("\"
returns=\"").append(m.getReturnType().getName()).append("\">\n");
            appendParameters(xml, m);
            xml.append("  </method>\n");
        }
        xml.append("</class>\n");
        return xml.toString();
    }

    /** Constructors and methods share the type Executable: parameters handled once. */
    private static void appendParameters(StringBuilder xml, Executable executable) {
        for (Parameter p : executable.getParameters()) {
            xml.append("    <parameter name=\"").append(p.getName())
               .append("\" type=\"").append(p.getType().getName()).append("\"/>\n");
        }
    }
}
```

stage02-reflection/src/main/java/sa4e/reflect/Inspector.java (abridged)

Three families of calls do all the work: `getDeclaredConstructors()`, `getDeclaredFields()` and `getDeclaredMethods()`. Each returns an array of member objects, and each member object can tell its name, its modifiers as a bit mask that `Modifier.toString` turns into `private static final`, and its types. Constructors and methods share a superclass, `Executable`, so parameters are handled once for both. Applied to `Candidate`, the result is:

```
<?xml version="1.0" encoding="UTF-8"?>
<class name="sa4e.reflect.Candidate" modifiers="public" superclass="java.lang.Object">
  <constructor modifiers="private">
    <parameter name="label" type="java.lang.String"/>
  </constructor>
  <constructor modifiers="public">
  </constructor>
  <constructor modifiers="public">
    <parameter name="x" type="int"/>
  </constructor>
  <field name="created" type="int" modifiers="private static"/>
  <field name="label" type="java.lang.String" modifiers="private final"/>
  <field name="x" type="int" modifiers="private"/>
  <method name="created" modifiers="public static" returns="int">
```

```
  </method>
  <method name="f" modifiers="public" returns="void">
    <parameter name="y" type="int"/>
    <parameter name="d" type="double"/>
  </method>
  <method name="getX" modifiers="public" returns="int">
  </method>
  <method name="reset" modifiers="protected" returns="void">
  </method>
  <method name="secret" modifiers="private" returns="java.lang.String">
  </method>
  <method name="toString" modifiers="public" returns="java.lang.String">
  </method>
</class>
```

Compare this with the `javap` listing at the end of stage 00. It is the same information, produced by a running program instead of a command-line tool. The private constructor, the private field, the private method: all there. Reflection does not respect visibility when *listing*; visibility only matters when you try to *use* a member, and even then, as we will see, only up to a point.

Declared versus inherited

The word "Declared" in those method names is not decoration. `getDeclaredMethods()` returns every method *declared in this class*, of any visibility, and nothing inherited. `getMethods()` returns every *public* method, including inherited ones, all the way up to `Object`:

Expression	Result
`Candidate.class.getDeclaredMethods().length`	`6 (all methods declared in Candidate, any visibility)`
`Candidate.class.getMethods().length`	`12 (public methods only, but including those inherited from Object)`
`Special.class.getDeclaredMethods().length`	`2 (compareTo plus a synthetic bridge method generated for Comparable<Special>)`

Six declared methods in `Candidate`: `created`, `f`, `getX`, `reset`, `secret`, `toString`. Twelve public methods: `created`, `f`, `getX`, `toString`, plus `equals`, `hashCode`, `getClass`, `notify`, `notifyAll` and three overloads of `wait` from `Object`. Neither view is "the" list of methods; which one you want depends on what you are doing. A serializer wants declared fields of the class *and* of its superclasses, so it walks the superclass chain from stage 01 and calls `getDeclaredFields()` on each.

The third line hides a small lesson. `Special` declares one method, `compareTo(Special)`, but the JVM sees two. Because `Comparable<T>` erases to `compareTo(Object)`, the compiler generates a *bridge* method with that signature that casts and delegates. It is marked `synthetic`, and `Inspector` skips synthetic members for that reason. Expect them; every generic class has some.

Parameter names, if you ask nicely

The XML above shows `<parameter name="y" type="int"/>`. That is less obvious than it looks:

Expression	Result
`parameter of f: y : int`	`isNamePresent=true`

```
parameter of f: d : double   isNamePresent=true
```

Parameter names are not in the class file by default. The JVM does not need them, so `javac` throws them away unless it is told to keep them with the flag `-parameters`. Without that flag, `Parameter.getName()` returns `arg0`, `arg1`, and `isNamePresent()` returns `false`. This project compiles with the flag because frameworks depend on it: Spring MVC maps a request parameter `?x=5` to a method parameter called `x` by name, and Jakarta REST and most JSON binders for constructors do the same. If you ever see `arg0` in an error message from such a framework, you now know which flag is missing.

Creating objects by name

The second half of the chapter stops looking and starts touching. First, construction:

```
Class<?> type = Class.forName("sa4e.reflect.Candidate");
Object viaPublic = type.getConstructor(int.class).newInstance(7);

Constructor<?> hidden = type.getDeclaredConstructor(String.class);
hidden.setAccessible(true);                       // private: this is the key
Object viaPrivate = hidden.newInstance("made by reflection");
```

Expression	Result
`type.getConstructor(int.class).newInstance(7)`	`Candidate{x=7, label='candidate#7'}`
`private constructor, after setAccessible(true)`	`Candidate{x=-1, label='made by reflection'}`

`getConstructor(int.class)` selects a constructor by its parameter types, exactly as the compiler would, and `newInstance(7)` calls it with boxed arguments. `getConstructor` sees only public constructors; `getDeclaredConstructor` sees all of them. And then the line that the whole chapter has been building up to: `setAccessible(true)`.

`setAccessible(true)` asks the reflection API to skip the language-level access check for this one member. After it, a private constructor can be called, a private method invoked, a private field read and written. It is the single most consequential method in `java.lang.reflect`, and every framework in the book's "In the wild" boxes calls it. It is also the method that Java 16 decided to say no to, in some circumstances, and we will get to that.

Calling methods by name

```
Candidate c = new Candidate(1);
Candidate.class.getMethod("f", int.class, double.class).invoke(c, 3, -1.0);

Method secret = Candidate.class.getDeclaredMethod("secret");
secret.setAccessible(true);
Object s = secret.invoke(c);

Method created = Candidate.class.getMethod("created");
Object n = created.invoke(null);                  // static: no target object
```

Expression	Result
`getMethod("f", int.class, double.class).invoke(c, 3, -1.0)`	`Candidate{x=3, label='candidate#1'}`
`private secret() after setAccessible(true)`	`the secret of candidate#1`
`static created(), invoked with target null`	`3`
`Integer.parseInt("forty-two") via reflection`	`InvocationTargetException wrapping java.lang.NumberFormatException: For input string: "forty-two"`
`secret.invoke("not a Candidate")`	`IllegalArgumentException: object of type java.lang.String is not an instance of sa4e.reflect.Candidate`

`Method.invoke(target, args...)` is the universal call. The target is the object to call the method on, or `null` for a static method. Arguments travel as `Object`, so `3` is boxed to an `Integer` on the way in and unboxed to an `int` by the JVM on the way out; the return value comes back boxed as well. Everything the compiler would have checked at compile time is now checked at run time, and the last two lines show what that looks like.

The fourth line is the one that bites everyone once. When the *called method* throws, `invoke` does not rethrow that exception. It wraps it in an `InvocationTargetException` whose `getCause()` is the real one. The reason is sound: `invoke` must distinguish "I could not perform the call" (wrong argument count, no access) from "the call ran and threw", and Java's checked-exception rules leave no other way. The consequence is that every piece of reflective code contains a `catch (InvocationTargetException e) { throw e.getCause(); }` somewhere, and so will ours in stage 04.

Reading and writing fields, even final ones

```
Field x = Candidate.class.getDeclaredField("x");
Field label = Candidate.class.getDeclaredField("label");
x.setAccessible(true);
label.setAccessible(true);

label.set(c, "changed although final");     // allowed on ordinary instance fields
Field counter = Candidate.class.getDeclaredField("created");
counter.setAccessible(true);
counter.setInt(null, 1000);                 // static field: target is null
```

Expression	Result
`x.getInt(c)`	`3`
`label.get(c)`	`candidate#1`
`label.set(c, "changed although final")`	`Candidate{x=3, label='changed although final'}`
`counter.setInt(null, 1000); Candidate.created()`	`1000`
`value.setInt(record, 2) on a record`	`IllegalAccessException: Can not set final int field sa4e.reflect.Variant$A.value to (int)2`

The third line should make you slightly uncomfortable. `label` is declared `private final`, and we changed it. For instance fields of ordinary classes, `setAccessible(true)` lifts the `final` as well as the `private`. This is not an oversight; serialization frameworks need it. When Jackson deserializes JSON into an object with final fields and no suitable constructor, this is how it does it, and `java.io.ObjectInputStream` has done the same since 1997.

The fifth line is where the JVM draws a line of its own. Fields of a *record* cannot be set this way, not even with `setAccessible(true)`; neither can `static final` fields or fields of hidden classes. Records promised immutability as part of their contract, and the JVM keeps that promise against reflection. A serializer that meets a record must go through the canonical constructor, which is exactly what `getRecordComponents()` from stage 01 makes easy.

The wall

Everything so far worked on our own classes. Now try it on the JDK:

```
Field stringValue = String.class.getDeclaredField("value");
stringValue.setAccessible(true);            // throws
```

Expression	Result
`String.class.getDeclaredField("value")`	`private final byte[] java.lang.String.value` (reading metadata always works)
`stringValue.setAccessible(true)`	`InaccessibleObjectException: Unable to make field private final byte[] java.lang.String.value accessible: module java.base does not "opens java.lang" to unnamed module @5b6f7412`

Listing the field works, as it always does; metadata is never hidden. But `setAccessible(true)` throws an `InaccessibleObjectException`, and the message tells the whole story: the module `java.base` does not *open* the package `java.lang` to our code.

This is *strong encapsulation*, introduced with the module system in Java 9 and enforced by default since Java 16. The JDK's own packages are closed to deep reflection unless a module explicitly opens them, and `java.base` does not. Before Java 16, the line above would have worked, and a dozen popular libraries used it to mutate strings, poke at `Unsafe` or rewrite class loaders. When Java 16 closed the door, those libraries broke, their authors complained, and the JDK team held firm, with one concession: the JVM flag

```
java --add-opens java.base/java.lang=ALL-UNNAMED ...
```

opens a package at launch time for code that genuinely needs it. If you have ever wondered why a Gradle build or an application server starts with a dozen `--add-opens` flags, this is why.

> **Note**
>
> Your own code can build the same wall. A `module-info.java` that does not `opens` a package makes that package's private members inaccessible to reflection from out-

> side, even with `setAccessible(true)`. Frameworks that need deep reflection ask you to `opens your.package to spring.core;`. Without a module descriptor, as in this project, your classes live in the “unnamed module”, which is open to everyone, which is why every example in this book just works.

Arrays

Arrays are classes, so of course there is reflection for them:

```
Object array = Array.newInstance(Candidate.class, 2);   // a Candidate[2]
Array.set(array, 0, new Candidate(5));
Array.set(array, 1, viaPrivate);
```

Expression	Result
`Array.newInstance(Candidate.class, 2)`	`Candidate[] of length 2`
`Array.get(array, 1)`	`Candidate{x=-1, label='made by reflection'}`

`java.lang.reflect.Array` creates and accesses arrays whose component type is a variable. It exists because `new T[n]` is impossible with erased generics, and because a deserializer that reads a JSON array has only a `Class<?>` to say what kind of array to build.

Invoker: a convenience layer

The raw API asks for exact parameter types: `getMethod("f", int.class, double.class)`. That is precise but tedious, and it is not what a scripting engine or a test framework wants. They have a name and some argument values, and they want the member that fits. `Invoker` is forty lines that do that:

```
public static Object call(Object target, String name, Object... args)
        throws ReflectiveOperationException {
    boolean isStatic = target instanceof Class<?>;
    Class<?> type = isStatic ? (Class<?>) target : target.getClass();
    for (Class<?> k = type; k != null; k = k.getSuperclass()) {
        for (Method m : k.getDeclaredMethods()) {
            if (m.getName().equals(name) && matches(m.getParameterTypes(), args)) {
                m.setAccessible(true);
                return m.invoke(isStatic ? null : target, args);
            }
        }
    }
    throw new NoSuchMethodException(type.getName() + "." + name);
}

/** True if every argument fits the corresponding parameter (boxing included). */
private static boolean matches(Class<?>[] parameterTypes, Object[] args) {
    if (parameterTypes.length != args.length) return false;
    for (int i = 0; i < args.length; i++) {
        if (args[i] == null) {
            if (parameterTypes[i].isPrimitive()) return false;   // null is no int
            continue;
        }
```

```
            if (!wrap(parameterTypes[i]).isInstance(args[i])) return false;
        }
        return true;
    }

    /** int.class -> Integer.class etc.; non-primitive types are returned unchanged. */
    private static Class<?> wrap(Class<?> type) {
        return type.isPrimitive() ? MethodType.methodType(type).wrap().returnType() : type;
    }
```

stage02-reflection/src/main/java/sa4e/reflect/Invoker.java (abridged)

Every piece of stage 01 is in there: the superclass walk, `isInstance`, `isPrimitive`. The `wrap` helper is a small trick worth stealing: `MethodType` from `java.lang.invoke` knows how to map `int.class` to `Integer.class`, so nobody has to write the eight-entry table by hand.

Expression	Result
`Invoker.newInstance("sa4e.reflect.Candidate", 9)`	`Candidate{x=9, label='candidate#9'}`
`Invoker.call(made, "secret")`	`the secret of candidate#9`
`Invoker.call(made, "f", 4, 1.5)`	`Candidate{x=-4, label='candidate#9'}`
`Invoker.call(Candidate.class, "created")`	`1002`
`Invoker.set(made, "x", 77); Invoker.get(made, "x")`	`77`
`Invoker.call(new Special(3), "secret")`	`the secret of candidate#3 (inherited private method)`

Notice the last line: `secret()` is private to `Candidate`, and `Invoker` found and called it on a `Special`. The compiler would never have allowed that. Notice also what we paid for the convenience: method names are now strings. The compiler cannot check them, and when someone renames `secret` to `hint`, the IDE's refactoring will not follow the string. Every framework that identifies members by name lives with this, which is one reason the next stage introduces annotations.

What does it cost?

The last section of stage 02 measures. Twenty million calls of `getX()` on four different objects, after three warm-up rounds so that the JIT compiler has done its work:

```
Method m = Candidate.class.getMethod("getX");
MethodHandle mh = MethodHandles.lookup()
        .findVirtual(Candidate.class, "getX", MethodType.methodType(int.class));

for (int i = 0; i < n; i++) sum += targets[i & 3].getX();              // direct
for (int i = 0; i < n; i++) sum += (int) m.invoke(targets[i & 3]);     // reflection
for (int i = 0; i < n; i++) sum += (int) mh.invokeExact(targets[i & 3]); // method handle
```

Call style	ns per call
direct call	0.35
`Method.invoke`	6.27
`MethodHandle.invokeExact`	2.37

These numbers are from one laptop and one run, with no benchmark harness, so treat them as a shape, not a measurement. The shape is reliable: a reflective call costs an order of magnitude more than a direct call, because arguments are boxed into an `Object[]`, access is checked, and the JIT cannot inline through `invoke`. The third row introduces `java.lang.invoke.MethodHandle`, a lower-level mechanism from Java 7 that is typed, does no boxing with `invokeExact`, and sits much closer to a direct call. Since Java 18, `Method.invoke` is itself implemented on top of method handles (JEP 416), which is why the gap is smaller than it used to be.

The practical rule is simple. Frameworks do reflection *once*, at startup, to find out what to do, and then do it millions of times through something cheaper: a cached `Method`, a `MethodHandle`, a generated class (stage 07). Reflection in a hot loop is a bug.

In the wild

- Jackson and Gson read and write private fields exactly as in the fields section, final ones included.
- JUnit finds and invokes test methods as in the methods section; stage 03 adds the annotations that tell it which ones.
- Spring and Jakarta CDI call constructors by reflection as in the construction section, with arguments they resolved themselves (stage 06).
- Mockito, Hibernate and most serialization frameworks needed `--add-opens` flags or rewrites when Java 16 enforced the wall.

The bill

- Member names are strings: no compile-time check, no refactoring support, errors at run time.
- A reflective call is several times slower than a direct one; cache members, use method handles, never reflect in a loop.
- Reflection defeats encapsulation. The JDK protects itself since Java 16; your own modules can do the same.
- GraalVM native image compiles with a closed world and cannot see the strings; everything accessed reflectively must be listed in `reachability-metadata.json`, or it is simply not there at run time.

Try it yourself

1. Let `Inspector` emit JSON instead of XML, and add the inherited public methods (`getMethods()`) as a separate section.
2. `Invoker.call(made, "f", 4, 1)` fails, because `1` is an `Integer` and the parameter is `double`. Add widening conversions.
3. Run stage 02 with `./run.sh 02 --add-opens java.base/java.lang=ALL-UNNAMED` and change the content of a `String` in place. Then explain why string literals elsewhere in the program may change too. (Hint: the string pool.)
4. Replace `Method.invoke` in the benchmark by a lambda created with `LambdaMetafactory` and compare again.

Takeaways

- Every member of a class is an object: `Constructor`, `Field`, `Method`, `Parameter`. Listing them ignores visibility; using them respects it, until `setAccessible(true)`.
- `invoke` wraps exceptions from the called method in `InvocationTargetException`; always unwrap.
- Final instance fields of ordinary classes can be changed; record fields, static finals and hidden classes cannot.
- The JDK is closed to deep reflection since Java 16; `--add-opens` is the escape hatch.
- Reflect once, cache, and call through something cheaper.

STAGE 03

Annotations

In which `Candidate` *finally gets to talk back, and we write two frameworks before lunch.*

Where we stand

Reflection, as of stage 02, is a one-way conversation. A framework can read everything about a class, but the class cannot say anything to the framework. Which field must never be null? Which method is a test? Which parameter has a valid range? Which constructor should a container use? In stage 02 such knowledge would have to live in the framework's code, in naming conventions (`testSomething`, `setSomething`) or in XML files kept somewhere else, going stale somewhere else.

Annotations are the answer: structured metadata, written *next to* the thing it describes, checked by the compiler, and, with the right retention, readable through reflection. The first half of this chapter is about the mechanism. The second half builds two small frameworks on it, a validator and a test runner, and in doing so introduces the idea that will carry the rest of the book: inversion of control.

Defining an annotation and reading it back

An annotation type is declared with `@interface`, and it is decorated with two meta-annotations that answer two questions: *where may it be written* and *how long does it live*:

```
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.CONSTRUCTOR, ElementType.FIELD,
         ElementType.METHOD, ElementType.PARAMETER})
public @interface Comment {
    String value() default "";
}
```

stage03-annotations/src/main/java/sa4e/reflect/Comment.java

`@Target` lists the kinds of elements that may carry the annotation; the compiler rejects `@Comment` on a local variable because `LOCAL_VARIABLE` is not in the list. `@Retention(RUNTIME)` says the annotation survives into the class file *and* is visible to reflection, which is what we want here. The body declares *members* as parameterless methods; `value()` is special in that `@Comment("text")` is shorthand for `@Comment(value = "text")`.

`Candidate` now wears a few of these:

```
@Comment("the class under investigation")
@Marker
public class Candidate {

    @Range(min = -100, max = 100)
    private int x;

    @NotNull
    private final String label;

    @Comment("reads x")
    public int getX() { return x; }

    public void f(@Range(min = -10, max = 10) int y, double d) { /* unchanged */ }
    // ...
}
```

Nothing about `Candidate`'s behaviour changed. Annotations are inert: they are claims, and nothing happens unless someone reads them. Reading is one method, `getAnnotation`, available on everything that can be annotated:

Expression	Result
`Candidate.class.getAnnotation(Comment.class).value()`	`the class under investigation`
`getX.getAnnotation(Comment.class).value()`	`reads x`
`Candidate.class.getMethod("f", ...).isAnnotationPresent(Comment.class)`	`false`
`Candidate.class.getMethod("reset")`	`NoSuchMethodException (getMethod sees public methods only; use getDeclaredMethod)`
`parameter y of f: getAnnotation(Range.class)`	`min=-10, max=10`

`Class`, `Constructor`, `Field`, `Method` and `Parameter` all implement `AnnotatedElement`, so the same `getAnnotation(Type.class)` works everywhere and returns `null` when the annotation is absent. The return type is the annotation type itself, so `.value()` is a normal, type-checked method call. Compare that with the stringly-typed `Invoker` of stage 02: annotations give the framework a vocabulary the compiler understands.

Meta-annotations: annotations all the way down

`@Retention` and `@Target` are themselves annotations, on the annotation type, and they can be read like any other:

Expression	Result
`Comment.class.isAnnotation()`	`true`
`Comment.class.getAnnotation(Retention.class).value()`	`RUNTIME`
`Comment.class.getAnnotation(Target.class).value()`	`[TYPE, CONSTRUCTOR, FIELD, METHOD, PARAMETER]`
`Comment.class.getDeclaredMethods()`	`[public abstract java.lang.String sa4e.reflect.Comment.value()]`

The last line is the one to remember: an annotation type is an *interface*, and its members are abstract methods. That has a consequence the next section exploits.

Inspector lists annotations without knowing any

Because members are methods, a tool can read an annotation it has never heard of. Stage 03's `Inspector` gets one new helper:

```
private static void appendAnnotations(StringBuilder xml, AnnotatedElement element,
                                      String indent) {
    for (Annotation a : element.getAnnotations()) {
        Class<? extends Annotation> type = a.annotationType();
        Method[] members = type.getDeclaredMethods();
        Arrays.sort(members, Comparator.comparing(Method::getName));
        xml.append(indent).append("<annotation type=\"")
           .append(type.getName()).append("\"");
        if (members.length == 0) { xml.append("/>\n"); continue; }
        xml.append(">\n");
        for (Method member : members) {
            Object value = member.invoke(a);          // read the member reflectively
            xml.append(indent).append("  <member name=\"").append(member.getName())
               .append("\">").append(escape(value)).append("</member>\n");
        }
        xml.append(indent).append("</annotation>\n");
    }
}
```

`Inspector` imports neither `Comment` nor `Range` nor `NotNull`. It asks the element for its annotations, asks each annotation for its type, asks the type for its methods, and *invokes* those methods on the annotation instance with the `Method.invoke` of stage 02. The output for `Candidate` now contains, for example:

```
<field name="x" type="int" modifiers="private">
  <annotation type="sa4e.reflect.Range">
    <member name="max">100</member>
    <member name="min">-100</member>
  </annotation>
</field>
<method name="f" modifiers="public" returns="void">
  <parameter name="y" type="int">
    <annotation type="sa4e.reflect.Range">
      <member name="max">10</member>
      <member name="min">-10</member>
    </annotation>
  </parameter>
  <parameter name="d" type="double"/>
</method>
```

This is also how an IDE, a documentation generator or a generic configuration dumper shows you annotations from libraries it has never seen.

Retention: who can see the annotation?

`RetentionPolicy` has three values, and most people have only ever consciously used one. Stage 03 defines three annotations that differ in nothing else, puts all three on a probe class, and asks who can see them:

```
@Retention(RetentionPolicy.SOURCE)  public @interface SourceOnly {}
@Retention(RetentionPolicy.CLASS)   public @interface ClassOnly {}     // the default
@Retention(RetentionPolicy.RUNTIME) public @interface RuntimeOnly {}

@SourceOnly @ClassOnly @RuntimeOnly
public static final class Probe {}
```

Expression	Result
`Retentions.Probe.class.getAnnotations()`	`[@sa4e.reflect.Retentions.RuntimeOnly()]`
`class file attribute RuntimeVisibleAnnotations`	`[Retentions$RuntimeOnly]`
`class file attribute RuntimeInvisibleAnnotations`	`[Retentions$ClassOnly]`

Reflection sees only `RuntimeOnly`. To find out what happened to the other two we do not take anybody's word for it; we read the bytes of `Probe.class` with the ClassFile API, which Java 24 added to the JDK and which stage 07 uses in earnest:

```
byte[] bytes = Retentions.Probe.class
        .getResourceAsStream("Retentions$Probe.class").readAllBytes();
ClassModel model = ClassFile.of().parse(bytes);
List<String> invisible = model.findAttribute(Attributes.runtimeInvisibleAnnotations())
        .map(a -> a.annotations().stream().map(x ->
x.classSymbol().displayName()).toList())
        .orElse(List.of());
```

`ClassOnly` is in the class file, in an attribute literally called `RuntimeInvisibleAnnotations`. `SourceOnly` is nowhere: `javac` saw it, could have acted on it, and threw it away. So:

- `SOURCE` is for the compiler and for annotation processors: `@Override`, `@SuppressWarnings`, Lombok's `@Getter`.
- `CLASS`, the *default* when `@Retention` is omitted, is for tools that read class files without running them: bytecode analysers, obfuscators, some build-time instrumentation. If you forget `@Retention` on your own annotation and then wonder why `getAnnotation` returns `null`, this is why.
- `RUNTIME` is for frameworks. Everything in this book is `RUNTIME`.

Repeatable and inherited

Two refinements that frameworks use and that surprise people when they first meet them.

```
@Repeatable(Tag.List.class)
@Retention(RetentionPolicy.RUNTIME)
public @interface Tag {
    String value();

    @Retention(RetentionPolicy.RUNTIME)
    @interface List { Tag[] value(); }
}

@Inherited
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Marker {}
```

```
@Tag("alpha") @Tag("beta")
static final class Tagged extends Candidate { /* ... */ }
```

Expression	Result
`Tagged.class.getAnnotation(Tag.class)`	`null`
`Tagged.class.getAnnotation(Tag.List.class)`	`@sa4e.reflect.Tag.List({@sa4e.reflect.Tag("alpha"), @sa4e.reflect.Tag("beta")})`
`Tagged.class.getAnnotationsByType(Tag.class)`	`[@sa4e.reflect.Tag("alpha"), @sa4e.reflect.Tag("beta")]`
`Tagged.class.getAnnotation(Marker.class)`	`@sa4e.reflect.Marker() (inherited: @Marker is @Inherited)`
`Tagged.class.getAnnotation(Comment.class)`	`null (not inherited: @Comment is not)`
`Tagged.class.getDeclaredAnnotation(Marker.class)`	`null (declared on Tagged itself: no)`

Writing `@Tag` twice is syntactic sugar. The compiler wraps the repeats into one container annotation, `@Tag.List`, which is why `getAnnotation(Tag.class)` finds nothing and `getAnnotation(Tag.List.class)` finds both. `getAnnotationsByType` unwraps the container for you; use it whenever an annotation is repeatable.

`@Inherited` makes a *class* annotation visible on subclasses: `Tagged` never says `@Marker`, but `getAnnotation(Marker.class)` finds the one on `Candidate`. `@Comment` is not `@Inherited`, so it stays with `Candidate`. Two fine points: `@Inherited` works only for class-level annotations, never for methods or fields, and `getDeclaredAnnotation` ignores inheritance altogether and reports only what is written on the class itself.

Annotations are interfaces; instances are proxies

One more look at `onClass`, the `@Comment` read from `Candidate`:

Expression	Result
`onClass.getClass().getName()`	`jdk.proxy2.$Proxy1`
`Proxy.isProxyClass(onClass.getClass())`	`true`
`onClass.annotationType()`	`interface sa4e.reflect.Comment`
`onClass instanceof Annotation`	`true`
`onClass.equals(getX.getAnnotation(Comment.class))`	`false`

Nobody wrote a class that implements `Comment`. Yet `onClass` is an object, and `onClass.value()` returns a string. The object's class is called `$Proxy1`, and it was generated by the JVM when the annotation was first read, using `java.lang.reflect.Proxy`. That mechanism is the subject of stage 04; for now, note that annotations are not magic either. Also note the last line: two annotations are `equals` when their members are equal, and these two have different `value`s.

Framework one: declarative validation

Now the payoff. `@NotNull` and `@Range` are annotations with `RUNTIME` retention and targets `FIELD` and `PARAMETER`. `Validator` is a class that knows nothing about `Candidate`, or about any other class it will ever check:

```
public static List<String> validate(Object target) throws IllegalAccessException {
    List<String> violations = new ArrayList<>();
    for (Class<?> k = target.getClass(); k != null; k = k.getSuperclass()) {
        for (Field f : k.getDeclaredFields()) {
            if (Modifier.isStatic(f.getModifiers())) continue;
            f.setAccessible(true);
            Object value = f.get(target);

            if (f.isAnnotationPresent(NotNull.class) && value == null) {
                violations.add(f.getName() + " must not be null");
            }
            Range range = f.getAnnotation(Range.class);
            if (range != null && value instanceof Number number) {
                long v = number.longValue();
                if (v < range.min() || v > range.max()) {
                    violations.add(f.getName() + " = " + v
                            + " is outside [" + range.min() + ", " + range.max() + "]");
                }
            }
        }
    }
    return violations;
}
```

stage03-annotations/src/main/java/sa4e/reflect/Validator.java

Every line is something from an earlier stage: the superclass walk, `getDeclaredFields`, `setAccessible`, `Field.get`, and now `getAnnotation`. Applied to a healthy and an unhealthy `Candidate`:

Expression	Result
`Validator.validate(new Candidate(7))`	`[]`
`Validator.validate(candidate with x=1000, label=null)`	`[x = 1000 is outside [-100, 100], label must not be null]`

(The unhealthy one had `label` set to `null` by reflection, because no constructor of `Candidate` allows it. Reflection breaks invariants as easily as it checks them.)

The point is where the knowledge lives. The rule "x must be between −100 and 100" sits on the field `x`, in `Candidate.java`, next to the thing it constrains. The code that enforces it is generic and shared. This is Jakarta Bean Validation in thirty lines; the real thing has hundreds of constraints, message interpolation, groups and cascading, but the shape is identical. Every web framework's form binding is this loop.

Framework two: a test runner

The second framework introduces the idea the rest of the book is built on. Three annotations:

```
@Retention(RUNTIME) @Target(METHOD) public @interface Test {}
@Retention(RUNTIME) @Target(METHOD) public @interface BeforeEach {}
@Retention(RUNTIME) @Target(METHOD)
public @interface Disabled { String value() default ""; }
```

a test class that uses them and never calls its own methods:

```
public class CandidateTests {
    private Candidate c;

    @BeforeEach
    void setUp() { c = new Candidate(7); }

    @Test
    void negativeDKeepsY() { c.f(3, -1.0); assertEquals(3, c.getX()); }

    @Test
    void positiveDNegatesY() { c.f(3, 1.0); assertEquals(-3, c.getX()); }

    @Test
    void resetSetsFortyTwo() { c.reset(); assertEquals(42, c.getX()); }

    @Test
    void deliberateFailure() { assertEquals(8, c.getX()); }   // wrong on purpose

    @Test @Disabled("waiting for stage 04")
    void notYet() {}

    private static void assertEquals(int expected, int actual) {
        if (expected != actual)
            throw new AssertionError("expected " + expected + " but was " + actual);
    }
}
```

and a runner:

```
public static List<Result> run(Class<?> testClass) throws ReflectiveOperationException {
    Method[] methods = testClass.getDeclaredMethods();
    Arrays.sort(methods, Comparator.comparing(Method::getName));   // no order promised
    List<Method> setups = new ArrayList<>();
    List<Method> tests = new ArrayList<>();
    for (Method m : methods) {
        m.setAccessible(true);                       // tests are usually package-private
        if (m.isAnnotationPresent(BeforeEach.class)) setups.add(m);
        if (m.isAnnotationPresent(Test.class)) tests.add(m);
    }

    List<Result> results = new ArrayList<>();
    for (Method test : tests) {
        Disabled disabled = test.getAnnotation(Disabled.class);
        if (disabled != null) {
            results.add(new Result(test.getName(), "skipped", disabled.value()));
            continue;
        }
        Object instance = testClass.getDeclaredConstructor().newInstance(); // fresh
        try {
            for (Method setup : setups) setup.invoke(instance);
            test.invoke(instance);
            results.add(new Result(test.getName(), "passed", ""));
        } catch (InvocationTargetException e) {                    // the test threw:
unwrap
```

```
            Throwable cause = e.getCause();
            results.add(new Result(test.getName(), "failed",
                    cause.getClass().getSimpleName() + ": " + cause.getMessage()));
        }
    }
    return results;
}
```

stage03-annotations/src/main/java/sa4e/reflect/TestRunner.java

Test	Outcome	Detail
`deliberateFailure`	failed	AssertionError: expected 8 but was 7
`negativeDKeepsY`	passed	
`notYet`	skipped	waiting for stage 04
`positiveDNegatesY`	passed	
`resetSetsFortyTwo`	passed	

Every mechanical detail is familiar by now: find methods, read annotations, create an instance, invoke, unwrap `InvocationTargetException`. What is new is the *direction of control*. `CandidateTests` is a class with five methods that nobody in `CandidateTests` ever calls. The runner decides what to create, in what order to call what, and what a failure means. The test class only *declares*, through annotations, which methods matter.

That reversal, where the framework calls your code instead of your code calling the framework, has a name: *inversion of control*, sometimes summarised as the Hollywood principle, "don't call us, we'll call you". JUnit is this runner with lifecycle extensions, parameter resolution and a reporting API. Servlet containers, message listeners, GUI event loops and, in stage 06, dependency-injection containers all work this way. Stage 03 is where the book's title quietly changes from "reflection" to "and more".

In the wild

- Jakarta Bean Validation (`@NotNull`, `@Min`, `@Max`, `@Size`) is the validator with hundreds of constraints and message interpolation.
- JUnit 5 is the test runner with extension points; `@BeforeEach`, `@Test` and `@Disabled` are its real names.
- JPA (`@Entity`, `@Column`), Jackson (`@JsonProperty`), Spring (`@Component`, `@Transactional`) and Jakarta REST (`@GET`, `@Path`) are all "framework reads annotation, framework acts". Stages 04 and 06 build two of them.
- Annotations with `CLASS` or `SOURCE` retention feed annotation processors at compile time (Lombok, MapStruct, Dagger), which generate code instead of reflecting at run time.

The bill

- Annotations are declarations, not behaviour. Nothing happens unless some code reads them; a forgotten framework call silently ignores every constraint, and the compiler cannot tell you.

- The same annotation can mean different things to different frameworks, and the order in which frameworks process them is expressed nowhere.
- Reading annotations is reflection, with stage 02's costs. Frameworks scan once at startup and cache.
- Forgetting `@Retention(RUNTIME)` is the single most common annotation bug, and its symptom is a silent `null`.

Try it yourself

1. Add `@Size(min, max)` for strings and collections to `Validator`.
2. Let `TestRunner` support `@AfterEach`, and run only the tests whose names are given on the command line.
3. Give `@Comment` a second member `author` with a default, and find out whether an annotation instance can tell which of its members were defaulted. (It cannot. Where could the information come from?)
4. Declare an annotation with `SOURCE` retention and verify with `javap -v` that it is absent from the class file; then do the same for `CLASS` retention and find it in `RuntimeInvisibleAnnotations`.

Takeaways

- An annotation is an interface with `@Target` and `@Retention`; its members are parameterless methods with optional defaults.
- Only `RUNTIME` retention is visible to reflection. `CLASS` is the default and is stored but invisible; `SOURCE` never leaves `javac`.
- `getAnnotation`, `isAnnotationPresent` and `getAnnotationsByType` work on every `AnnotatedElement`; annotation instances are generated proxies.
- A framework is a loop over reflection plus annotations: find the annotated elements, then act. The validator and the test runner are that loop.
- When the framework calls your code, that is inversion of control.

STAGE 04

Dynamic proxies

In which an object implements an interface nobody wrote a class for, and we build an EJB container in three handlers.

Where we stand

Stage 03 ended with a `@Transactional`-shaped hole. Annotations can *declare* that something should happen around a method call, but somebody has to make it happen, and the obvious way does not scale. To log every call on an interface you write a decorator class that implements the interface and forwards every method. To add transactions you write another. To add security, caching, metrics, retries, you write another each. And you do it again for every interface. The number of classes is the product of interfaces and concerns, and every one of them is boilerplate that forwards calls.

Stage 04 replaces all of them with a single mechanism. `java.lang.reflect.Proxy` generates, at run time, a class that implements any interface you name, and routes every call on it to one method of your choosing. From there, logging, validation, transactions, lazy loading and even remote calls are each one short class.

The pain, concretely

Stage 04 gives `Candidate` an interface, because proxies need one:

```
public interface Capability {
    int getX();
    void f(int y, double d);
}

public class Candidate implements Capability {
    // ...
    @Override
    @Transactional
    public void f(@Range(min = -10, max = 10) int y, double d) { /* unchanged */ }
}
```

and shows the hand-written way once, so that the next section has something to beat:

```
static final class LoggingCapability implements Capability {
    private final Capability target;
```

```
    private final List<String> log;

    LoggingCapability(Capability target, List<String> log) {
        this.target = target;
        this.log = log;
    }

    @Override public int getX() {
        log.add("getX()");
        return target.getX();
    }

    @Override public void f(int y, double d) {
        log.add("f(" + y + ", " + d + ")");
        target.f(y, d);
    }
}
```

Expression	Result
`new LoggingCapability(new Candidate(7), log).f(3, -1.0); getX()`	`3`
`log`	`[f(3, -1.0), getX()]`

It works. It also knows `Capability` by name, mentions both of its methods, and would need to be rewritten for every other interface and every other concern. Multiply by a real project's fifty interfaces and six cross-cutting concerns, and the decorator pattern becomes three hundred classes of pure forwarding.

One handler for all interfaces

```
public final class LoggingHandler implements InvocationHandler {
    private final Object target;
    private final List<String> log;

    public LoggingHandler(Object target, List<String> log) {
        this.target = target;
        this.log = log;
    }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        String call = Proxies.signature(method, args);          // "f(3, -1.0)"
        log.add("→ " + call);
        try {
            Object result = Proxies.invokeTarget(target, method, args);
            boolean isVoid = method.getReturnType() == void.class;
            log.add("← " + call + (isVoid ? "" : " = " + result));
            return result;
        } catch (Throwable t) {
            log.add("✗ " + call + " threw " + t.getClass().getSimpleName()
                    + ": " + t.getMessage());
            throw t;
        }
    }
}
```

stage04-proxies/src/main/java/sa4e/reflect/LoggingHandler.java

```
Candidate real = new Candidate(7);
Capability proxy = Proxies.wrap(Capability.class, new LoggingHandler(real, log));
proxy.f(3, -1.0);
```

where `Proxies.wrap` is the one line that matters in the whole chapter:

```
public static <T> T wrap(Class<T> type, InvocationHandler handler) {
    return (T) Proxy.newProxyInstance(type.getClassLoader(), new Class<?>[]{type},
handler);
}
```

Expression	Result
`proxy.f(3, -1.0); proxy.getX()`	`3`
`proxy.toString()`	`Candidate{x=3, label='candidate#7'}`
`log`	`[→ f(3, -1.0), ← f(3, -1.0), → getX(), ← getX() = 3, → toString(), ← toString() = Candidate{...}]`
`proxy.getClass().getName()`	`jdk.proxy1.$Proxy0`
`Proxy.isProxyClass(proxy.getClass())`	`true`
`proxy.getClass().getInterfaces()`	`[interface sa4e.reflect.Capability]`
`proxy instanceof Capability`	`true`
`proxy instanceof Candidate`	`false`

Read the handler and the results together. `LoggingHandler` does not mention `Capability`, `getX` or `f`. It has one method, `invoke`, and the JVM calls that method for *every* method of *every* interface the proxy implements, passing the `Method` object from stage 02 and the arguments as an `Object[]`. The handler logs, forwards with `Method.invoke`, logs again, and returns whatever came back. The same handler class would log calls on `java.sql.Connection` or on `Runnable` without a single change.

`Proxy.newProxyInstance` is where the class comes from. Given a class loader, a list of interfaces and a handler, it generates a class at run time, in a package called `jdk.proxy1`, with a name like `$Proxy0`, that implements the interfaces and whose every method is a stub calling `handler.invoke(this, method, args)`. Note `proxy instanceof Candidate` is `false`: the proxy is not a `Candidate` wearing a disguise, it is a brand-new class that happens to implement the same interface. That is why the interface was needed.

And note that `toString()` went through the handler too. The proxy class overrides `equals`, `hashCode` and `toString` and routes them to the handler like everything else, which is sometimes exactly what you want (here the log shows it) and sometimes a surprise (a handler that forwards to a remote server will make `System.out.println(proxy)` do a network round trip).

What the generated class looks like

We have an `Inspector`. Let us point it at the proxy class:

```
<class name="jdk.proxy1.$Proxy0" modifiers="public final"
       superclass="java.lang.reflect.Proxy">
  <constructor modifiers="public">
    <parameter name="arg0" type="java.lang.reflect.InvocationHandler"/>
```

```
  </constructor>
  <field name="m0" type="java.lang.reflect.Method" modifiers="private static final"/>
  <field name="m1" type="java.lang.reflect.Method" modifiers="private static final"/>
  <field name="m2" type="java.lang.reflect.Method" modifiers="private static final"/>
  <field name="m3" type="java.lang.reflect.Method" modifiers="private static final"/>
  <field name="m4" type="java.lang.reflect.Method" modifiers="private static final"/>
  <method name="equals" modifiers="public final" returns="boolean">
    <parameter name="arg0" type="java.lang.Object"/>
  </method>
  <method name="f" modifiers="public final" returns="void">
    <parameter name="arg0" type="int"/>
    <parameter name="arg1" type="double"/>
  </method>
  <method name="getX" modifiers="public final" returns="int">
  </method>
  <method name="hashCode" modifiers="public final" returns="int">
  </method>
  <method name="toString" modifiers="public final" returns="java.lang.String">
  </method>
</class>
```

Everything about the mechanism is visible. The class is `final` and extends `java.lang.reflect.Proxy`, which holds the handler in a field `h`. Its single constructor takes the handler. It caches the five `Method` objects it will pass to the handler (two from `Capability`, three from `Object`) in static fields `m0` to `m4`, so that `invoke` does not have to look them up on every call. The parameter names are `arg0`, `arg1`: the generated class was not compiled with `-parameters`, because it was not compiled at all. Stage 07 writes a class like this by hand.

A chain of handlers

Handlers compose, because a handler's target can itself be a proxy. Stage 04 adds two more handlers and stacks them:

```
TransactionLog tx = new TransactionLog();
Capability validated = Proxies.wrap(Capability.class,
        new ValidatingHandler(real));
Capability transactional = Proxies.wrap(Capability.class,
        new TransactionalHandler(validated, tx));
Capability chained = Proxies.wrap(Capability.class,
        new LoggingHandler(transactional, log));
```

```
caller ──▶ LoggingHandler ──▶ TransactionalHandler ──▶ ValidatingHandler ──▶ Candidate
            ($Proxy2)             ($Proxy1)               ($Proxy0)          (real)
```

`ValidatingHandler` checks `@NotNull` and `@Range` on the *parameters* of the implementing method before the call goes through, the stage 03 validator moved from after-the-fact to the entrance. `TransactionalHandler` is the one everyone has been waiting for since stage 03:

```
@Override
public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
    Method implementation = Proxies.implementationMethod(target, method);
    if (!implementation.isAnnotationPresent(Transactional.class)) {
        return Proxies.invokeTarget(target, method, args);       // not our business
    }
```

```
    int tx = transactions.begin(Proxies.signature(method, args));
    try {
        Object result = Proxies.invokeTarget(target, method, args);
        transactions.commit(tx);
        return result;
    } catch (Throwable t) {
        transactions.rollback(tx, t);
        throw t;
    }
}
```

stage04-proxies/src/main/java/sa4e/reflect/TransactionalHandler.java

`TransactionLog` stands in for a transaction manager and merely records what it was asked to do. One good call and one bad call through the chain:

Expression	Result
`chained.f(5, 1.0); chained.getX()`	`-5`
`chained.f(1000, 1.0)`	`IllegalArgumentException: y = 1000 is outside [-10, 10]`
`real.getX() afterwards`	`-5`
`log`	`[→ f(5, 1.0), ← f(5, 1.0), → getX(), ← getX() = -5, → f(1000, 1.0), ✗ f(1000, 1.0) threw IllegalArgumentException: y = 1000 is outside [-10, 10]]`
`tx.entries()`	`[begin tx1 for f(5, 1.0), commit tx1, begin tx2 for f(1000, 1.0), rollback tx2 because of IllegalArgumentException]`

Follow the second call through the picture. The logging handler records the entry and forwards. The transactional handler sees `@Transactional` on `f`, begins `tx2` and forwards. The validating handler reads `@Range(min = -10, max = 10)` on parameter `y`, finds 1000, and throws. The exception travels back: the transactional handler rolls back `tx2` and rethrows, the logging handler records the failure and rethrows, the caller catches an `IllegalArgumentException`. `Candidate` was never reached, its `x` is still `-5`, and `Candidate` contains not one line about logging, validation or transactions.

That is an EJB container. That is Spring AOP. `getX` is not `@Transactional`, so it passes the middle handler untouched; a real container would also have security, metrics and a dozen other handlers in the chain, each deciding for itself whether a call is its business.

A detail that bit us

`Proxies.invokeTarget` deserves its own paragraph, because the first version of this stage got it wrong:

```
public static Object invokeTarget(Object target, Method method, Object[] args)
        throws Throwable {
    try {
        return method.invoke(target, args);
    } catch (InvocationTargetException e) {
        throw e.getCause();                          // essential: unwrap
    }
}
```

Stage 02 warned that `Method.invoke` wraps exceptions in `InvocationTargetException`. In a handler, forgetting to unwrap has a nasty consequence: the proxy's generated method sees

a checked exception that the interface method did not declare and wraps it *again*, in an `UndeclaredThrowableException`. The caller then catches an `UndeclaredThrowableException` wrapping an `InvocationTargetException` wrapping the `IllegalArgumentException` it actually wanted. Every proxy-based framework has this unwrapping in one central place.

Where do the annotations live?

The second thing the first version of this stage got wrong is worth a section of its own, because it is a question every framework author has had to answer:

Expression	Result
`Capability.class.getMethod("f", ...).isAnnotationPresent(Transactional.class)`	`false`
`Candidate.class.getMethod("f", ...).isAnnotationPresent(Transactional.class)`	`true`
`Proxies.unwrap(chained) == real`	`true`
`Proxies.unwrap(chained).getClass()`	`Candidate`

The `Method` object a handler receives belongs to the *interface*, `Capability.f`, and there is no `@Transactional` on the interface. The annotation is on the implementation, `Candidate.f`. So a handler must look up the method with the same name and parameter types on the class of the real object. And in a chain, the handler's target is not the real object but the next proxy, whose class is `$Proxy0` and carries no annotations either. The first version of `TransactionalHandler` looked there, found nothing, and the transaction log stayed empty.

The fix is `Proxies.unwrap`, which follows the chain down:

```
public static Object unwrap(Object object) {
    while (Proxy.isProxyClass(object.getClass())
            && Proxy.getInvocationHandler(object) instanceof Forwarding forwarding) {
        object = forwarding.target();
    }
    return object;
}
```

Handlers that forward implement a tiny `Forwarding` interface with one method, `target()`, so that the chain can be walked. Spring has the same method, `AopUtils.getTargetClass`, for the same reason, and its documentation warns, as ours now does, that annotations on interfaces are not always honoured.

A virtual proxy

Not every handler forwards to an object that already exists:

```
public final class LazyHandler implements InvocationHandler {
    private final Supplier<?> factory;
    private Object target;

    public LazyHandler(Supplier<?> factory) { this.factory = factory; }

    @Override
```

```
    public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        if (target == null) target = factory.get();   // first call creates the target
        return Proxies.invokeTarget(target, method, args);
    }
}
```

Expression	Result
`after creating the lazy proxy: lazy.isInitialized()`	`false`
`Candidate.created() unchanged`	`true`
`lazyProxy.getX()`	`9`
`now lazy.isInitialized()`	`true`

This is the *virtual proxy* of the Gang of Four book, and it is how a JPA provider implements lazy loading. When you load an `Order`, its `customer` field holds a proxy, not a `Customer`; the database is queried the moment the first getter is called. It is also why `LazyInitializationException` exists: if the first getter is called after the session that could run the query has been closed, the factory has nowhere to go.

A remote stub

The last handler does not forward to an object at all. It forwards to *somewhere else*:

```
public static <T> T create(Class<T> type, Function<String, String> transport,
                           List<String> wireLog) {
    return Proxies.wrap(type, (proxy, method, args) -> {
        if (method.getDeclaringClass() == Object.class) {        // toString etc. stay
local
            return "RemoteStub[" + type.getSimpleName() + "]";
        }
        String request = marshal(method, args);      // "f|int,double|int:2|double:-1.0"
        wireLog.add("request: " + request);
        String reply = transport.apply(request);
        wireLog.add("reply:   " + reply);
        return unmarshal(reply);                                  // "ok|int:2" -> 2
    });
}
```

stage04-proxies/src/main/java/sa4e/reflect/RemoteStub.java (abridged)

On the other side of the transport sits a `Skeleton`, which does the reverse: split the request, map the type names back to classes, find the method with `getMethod(name, types)`, invoke it on the real object, and marshal the result or the error. For the demo the transport is a method call, `server::handle`, but it could be a socket or an HTTP client without changing a line of the stub.

Expression	Result
`stub.f(2, -1.0); stub.getX()`	`2`
`stub.f(1000, 1.0)`	`remote: IllegalArgumentException: y = 1000 is outside [-10, 10]`
`stub.toString()`	`RemoteStub[Capability]`
`wire`	`[request: f|int,double|int:2|double:-1.0, reply: ok|void, request: getX|, reply: ok|int:2, request: f|`

```
                                int,double|int:1000|double:1.0, reply: error|
                                IllegalArgumentException: y = 1000 is outside [-10, 10]]
```

This is Java RMI. It is also a CORBA stub, a SOAP client proxy, and what Feign and Retrofit do with annotated interfaces over HTTP. The toy wire format, `type:value` joined by bars, stands in for XDR, IIOP, JSON or Protocol Buffers; the shape of the mechanism is identical. And look at what the skeleton's real object was in the demo: `chained`, the interceptor chain from before. The third request was rejected by the validating handler on the server side, rolled back by the transactional handler, and the error travelled back over the wire. Stub, skeleton, container services, bean. That picture is the whole of lecture 5 in one line of code.

Pitfalls

Expression	Result
`handler throws IOException, getX() declares none`	`UndeclaredThrowableException wrapping java.io.IOException: checked exception from the handler`
`Proxies.wrap(Candidate.class, ...)`	`IllegalArgumentException: sa4e.reflect.Candidate is not an interface`

- **Interfaces only.** A JDK proxy cannot stand in for a class. Frameworks that must intercept a class without an interface generate a subclass instead, with CGLIB or ByteBuddy; stage 07 does it by hand.
- **Self-invocation.** If `Candidate.f` called `this.getX()`, that call would go straight to the real object and bypass every handler. The proxy only sees calls that come from outside. Spring's most frequently asked `@Transactional` question is exactly this: "why is my transaction not started when one method of my service calls another?"
- **Identity.** `proxy != real`, `proxy.getClass() != Candidate.class`, and `equals` goes through the handler. Collections of proxies, caches keyed by object and `instanceof` checks all need thought.
- **Checked exceptions.** A handler that throws a checked exception the interface method does not declare produces an `UndeclaredThrowableException`. Unwrap `InvocationTargetException`, and only throw what the interface allows.
- **Stack traces.** Every call now has `$Proxy0.f`, `LoggingHandler.invoke`, `Method.invoke` and a few JDK frames in it. The breakpoint in `Candidate.f` is still hit; the question "who called me?" has a longer answer.

In the wild

- EJB containers and Spring AOP: the handler chain is container-managed transactions, declarative security, interceptors and aspects. Spring creates exactly these JDK proxies for beans that implement interfaces, and CGLIB subclasses otherwise.
- Java RMI, CORBA, SOAP client proxies, Feign, Retrofit, gRPC stubs: the remote stub with a real transport and a real marshalling format.

- Hibernate and other JPA providers: the virtual proxy, for lazy associations.
- Annotation instances (stage 03), `java.sql.Connection` wrappers from connection pools, Mockito mocks of interfaces, and `java.lang.reflect.Proxy`'s own test suite.

The bill

- One reflective call per invocation, with stage 02's overhead; proxies are not for hot inner loops.
- Interfaces only; subclass generation is a separate, heavier mechanism.
- Self-invocation bypasses everything, `equals` / `hashCode` / `getClass` lie a little, and stack traces grow.
- Every handler is a place where behaviour happens that is written nowhere near the code it affects. Debugging "why did this call take 200 ms" now means reading the chain.

Try it yourself

1. Add a `CachingHandler` that remembers the result of `getX()` until the next call of `f`.
2. Let `LoggingHandler` measure each call in microseconds and log it.
3. Extend `Wire` and `Skeleton` so that methods can take and return `String` arrays, then write down what would have to change for arbitrary objects. (You are designing a serialization format. Stop when you notice.)
4. Make the stub talk over a real `java.net.ServerSocket` in a second thread.

Takeaways

- `Proxy.newProxyInstance` generates a class that implements interfaces and routes every call to `InvocationHandler.invoke(proxy, method, args)`.
- One handler serves every interface; concerns become handlers, and handlers chain.
- Always unwrap `InvocationTargetException`; always look for annotations on the real class, through the chain.
- A virtual proxy creates its target on first use; a remote stub has no local target at all.
- Interfaces only, and self-invocation bypasses the proxy. Both limits shape how frameworks ask you to write code.

STAGE 05

Dynamic class loading

In which classes arrive while the program is running, a class turns out to be not quite the same as itself, and we compile Java from inside Java.

Where we stand

Everything so far, proxies included, has worked on classes that existed when `javac` ran. Stage 01's `Class.forName` could turn a string into a class, but the class had to be on the class path from the start. A new implementation of `Capability` meant recompiling and redeploying the whole program.

Enterprise systems need the opposite. Choose an implementation by configuration. Discover implementations that were dropped into a directory. Add a plugin after deployment. Replace a component while the system runs. All of this rests on one piece of the JVM that most Java developers never touch directly: the class loader.

Every class has a loader

A class is not loaded by "the JVM". It is loaded by a specific object, an instance of `java.lang.ClassLoader`, and the class remembers which one:

Expression	Result
`String.class.getClassLoader()`	`null (null = bootstrap loader, part of the JVM)`
`java.sql.Driver.class.getClassLoader()`	`PlatformClassLoader "platform"`
`Candidate.class.getClassLoader()`	`AppClassLoader "app"`
`ClassLoader.getSystemClassLoader()`	`AppClassLoader "app"`
`parent chain of the application loader`	`app -> platform -> bootstrap`

Three loaders exist before your program starts. The *bootstrap* loader is part of the JVM itself, written in C++, and loads `java.base`: `String`, `Object`, `Class`. It has no Java object, which is why `String.class.getClassLoader()` is `null`. The *platform* loader loads the rest of the JDK, `java.sql` for instance. The *application* loader, also called the system loader, loads your class path: `Candidate`, `Main`, every jar you added.

They form a chain, and the chain has a rule: *parent first*. When the application loader is asked for a class, it asks the platform loader first, which asks the bootstrap loader first. Only if the parents do not

have the class does a loader look in its own sources. This is why you cannot replace `java.lang.String` by putting a class of that name on your class path: the bootstrap loader answers first, every time.

Loading is not initializing

Before adding loaders, a subtlety about the ones we have. A class goes through three phases: *loading* (reading the bytes and creating the `Class` object), *linking* (verification, preparation of static fields with default values), and *initialization* (running the static initializers). They do not happen at the same time, and the gap is observable. Stage 05 adds a class that leaves a trace when it is initialized:

```
public final class Noisy {
    static {
        LoadEvents.EVENTS.add("static initializer of Noisy ran");
    }
    public static final int CONSTANT = 42;   // compile-time constant: inlined by javac
    public static int counter = 1;           // a real static field
}
```

Expression	Result
`Class.forName("sa4e.reflect.Noisy", false, loader)`	`class sa4e.reflect.Noisy`
`LoadEvents.EVENTS after loading`	`[]`
`noisy.getDeclaredFields().length (reflection on an uninitialized class)`	`2`
`LoadEvents.EVENTS after reflection`	`[]`
`Noisy.CONSTANT (a compile-time constant, inlined by javac)`	`42`
`LoadEvents.EVENTS after reading the constant`	`[]`
`Noisy.counter (a real static field)`	`1`
`LoadEvents.EVENTS after reading the field`	`[static initializer of Noisy ran]`

The three-argument `Class.forName(name, false, loader)` loads without initializing. Reflection on the class works; `getDeclaredFields()` needs only the metadata. Reading `Noisy.CONSTANT` does not initialize either, for a reason that surprises people: `javac` saw a `static final int` with a constant value and inlined `42` into `Main`, so `Noisy` is not even consulted. Only `Noisy.counter`, a real static field, triggers initialization, and only then does the static block run.

The JVM initializes a class at its first *active use*: `new`, a static method call, a non-constant static field access. The one-argument `Class.forName(name)` counts as active use and initializes immediately, which brings us back to the JDBC idiom from stage 01: `Class.forName("org.postgresql.Driver")` was never about loading the driver, it was about running its static initializer, in which the driver registered itself with `DriverManager`. Stage 06's `ClassScanner` uses the three-argument form for the opposite reason: it looks at hundreds of classes and must not run any of their initializers.

An implementation chosen by configuration

Now the first of three ways for code to arrive late. Stage 05 adds a second implementation of `Capability`, called `Alternative`, and a properties file:

```
capability.impl=sa4e.reflect.Candidate
```

```
Properties config = new Properties();
config.load(Main.class.getResourceAsStream("/application.properties"));
String name = System.getProperty("capability.impl",
config.getProperty("capability.impl"));

Capability chosen = Class.forName(name)
        .asSubclass(Capability.class)
        .getDeclaredConstructor()
        .newInstance();
```

Expression	Result
`application.properties`	`capability.impl=sa4e.reflect.Candidate`
`effective name (overridable with -Dcapability.impl=...)`	`sa4e.reflect.Candidate`
`chosen.getClass()`	`Candidate`
`chosen.f(3, 4.0); chosen`	`Candidate{x=-3, label='candidate#0'}`
`the same with sa4e.reflect.Alternative`	`Alternative{x=7}`

`Main` mentions neither `Candidate` nor `Alternative`. The name comes from a file, or from the command line (`./run.sh 05 -Dcapability.impl=sa4e.reflect.Alternative` prints `Alternative{x=-3}` instead), and the only new call is `asSubclass`. It checks that the configured class really implements `Capability` and returns a `Class<? extends Capability>`, so that the result of `newInstance()` is typed without a cast. If the configuration names a class that does not implement the interface, `asSubclass` throws a `ClassCastException` at the point where the mistake can still be explained; without it, the cast would fail later and further away.

This is the oldest form of dependency injection, in use since before the term existed: JDBC drivers, JNDI providers, XML parser factories and logging back-ends were all selected by a class name in a system property.

ServiceLoader: the JDK's own plugin registry

Configuration by class name requires somebody to write the name down. Often the better question is "which implementations are *available*?", and the JDK has had an answer since Java 6:

```
META-INF/services/sa4e.reflect.Capability
```

```
# Implementations of sa4e.reflect.Capability, found by java.util.ServiceLoader.
sa4e.reflect.Candidate
sa4e.reflect.Alternative
```

The file is named after the interface and lists implementations, one per line. Any jar on the class path can contribute such a file, and `ServiceLoader` collects them all:

```
List<String> providers = ServiceLoader.load(Capability.class).stream()
        .map(p -> p.type().getName()).toList();       // type() does not instantiate

for (Capability c : ServiceLoader.load(Capability.class)) {   // iterating instantiates
    c.f(1, 1.0);
}
```

Expression	Result
`ServiceLoader.load(Capability.class).stream()`	`[sa4e.reflect.Candidate, sa4e.reflect.Alternative]`
`instances after f(1, 1.0)`	`[Candidate{x=-1, label='candidate#0'}, Alternative{x=2}]`

Underneath, `ServiceLoader` does what stage 01 did: it finds the resource files with `getResources`, reads the names, and calls `Class.forName` and the public no-arg constructor. The value is in the convention. A JDBC driver jar contains `META-INF/services/java.sql.Driver`, and `DriverManager` finds it without anybody calling `Class.forName` any more. Charset providers, file system providers, JAX-RS runtimes, SLF4J back-ends and JUnit engines are all found this way, and with the module system the same thing is spelled `provides Capability with Candidate;` in `module-info.java`.

A plugin compiled while the program runs

Both mechanisms so far still needed the class on the class path. The third does not. Stage 05 ships a source file that the build deliberately does not compile:

```
// This file is NOT compiled by the build. Stage 05 compiles it while the program runs.
public class Plugin implements sa4e.reflect.Capability {
    private int x;
    public int getX() { return x; }
    public void f(int y, double d) { x = y * 2; }
    public String toString() { return "Plugin version 1 {x=" + x + "}"; }
}
```

stage05-loading/plugins/Plugin.java

Every JDK contains the compiler as a library, `javax.tools.JavaCompiler`, and calling it is a few lines:

```
JavaCompiler compiler = ToolProvider.getSystemJavaCompiler();   // null on a bare JRE
Files.writeString(outputDir.resolve("Plugin.java"), source);
int status = compiler.run(null, null, errors,
        "-classpath", System.getProperty("java.class.path"),   // Capability must resolve
        "-d", outputDir.toString(),
        outputDir.resolve("Plugin.java").toString());
```

Now the class file exists in a temporary directory that no loader knows about. So we make a loader that does:

```
URL[] urls = {classesDir.toUri().toURL()};
URLClassLoader loader = new URLClassLoader("plugin:v1", urls,
        PluginLoader.class.getClassLoader());   // parent: the application loader
Class<?> pluginClass = loader.loadClass("Plugin");
Capability plugin = (Capability) pluginClass.getDeclaredConstructor().newInstance();
```

Expression	Result

`plugin.f(5, 0.0); plugin`	`Plugin version 1 {x=10}`
`pluginClass.getClassLoader()`	`URLClassLoader "plugin:v1"`
`Capability.class.getClassLoader()`	`AppClassLoader "app"`
`pluginClass.getInterfaces()[0] == Capability.class`	`true`

Read the last three lines together; they are the whole theory of plugins. `Plugin` was loaded by the new `URLClassLoader`. Its interface `Capability` was not: the new loader asked its parent first, the parent is the application loader, and the application loader already had `Capability`. So the `Capability` the plugin implements is *the same* `Class` *object* the host program uses, and the cast to `Capability` succeeds. The shared interface, loaded by a shared ancestor, is the contract between host and plugin. Everything the plugin defines itself lives in its own loader and could be thrown away with it.

Two versions, side by side

Here is the experiment that explains a famous error message. Compile a second version of the same source, with two strings changed, into a second directory, and load it with a second loader:

Expression	Result
`second version compiled and loaded; plugin2`	`Plugin version 2 {x=15}`
`pluginClass.getName().equals(pluginClass2.getName())`	`true`
`pluginClass == pluginClass2`	`false`
`pluginClass.isInstance(plugin2)`	`false`
`loading version 1 once more with a third loader: again == pluginClass`	`false`
`pluginClass.cast(plugin2)`	`ClassCastException: Cannot cast Plugin to Plugin`

Both classes are called `Plugin`. They are different classes. A class on the JVM is identified by its *name and its defining loader*, and two loaders that read the same bytes still produce two unrelated classes, as the fifth line shows. Stage 01 said there is exactly one `Class` object per loaded class; this is the fine print. “Cannot cast Plugin to Plugin” is the message you get when an object from one loader is handed to code that expects the same class from another loader, and once you have seen this experiment you will never be puzzled by it again.

The same fact is a feature. An application server can run two web applications that bundle different versions of the same library, because each application has its own loader and the two copies of `org.apache.commons.Whatever` never meet. And it can *redeploy* an application by discarding its loader and creating a new one: the old classes become garbage as soon as nothing references them, the loader, or any of their instances. Version 2 of `Plugin` did not replace version 1 in the demo, it joined it; the host chooses which one to talk to.

Consequences

Note

Loading a class runs its static initializer. That makes “load a class from an untrusted location” exactly equivalent to “run untrusted code”. Log4Shell, the vulnerability of December 2021, was nothing more than Log4j letting a log message contain a JNDI lookup, and JNDI fetching a class from a server the attacker named. The whole mechanism of this chapter was in place; only the trust was missing.

- **Flexibility.** Implementations can be swapped by configuration, discovered by registration, or added after deployment, without touching the host.
- **Isolation.** Separate loaders let incompatible versions coexist and let components be unloaded, at the price of `ClassCastException` and `LinkageError` when objects cross a boundary they were not meant to cross.
- **Leaks.** A loader stays alive as long as anything references any of its classes. One lingering thread or one cached object keeps an entire redeployed application in memory; this was the classic application-server leak for a decade.
- **Closed worlds.** GraalVM native image decides at build time which classes exist. Nothing in this chapter is possible there unless the classes are known in advance.

In the wild

- JDBC drivers, charset providers, `java.util.logging` and SLF4J back-ends, JAX-RS runtimes and JUnit engines are discovered with `ServiceLoader` or `Class.forName` from configuration.
- Tomcat, Jetty, WildFly and every other application server give each deployed application its own loader, with parent-first delegation deliberately broken for the application’s own libraries.
- OSGi and the Java Platform Module System add versioning and visibility rules on top of loaders.
- JSP engines, Groovy, the Java shell and IDE “evaluate expression” features compile source at run time like section five; hot swap in a debugger is the JVM-level cousin.

The bill

- Security: a loaded class is executed code. Never load from a location you do not trust.
- `ClassCastException` across loaders, `LinkageError` when the same class is visible twice, memory leaks when an old loader is kept alive.
- Everything that uses `Class.forName` is invisible to static analysis and to closed-world compilers.
- Run-time compilation needs a JDK, not a JRE, and costs seconds.

Try it yourself

1. Run `./run.sh 05 -Dcapability.impl=sa4e.reflect.Alternative`, then with a class that does not exist and with one that exists but does not implement `Capability`. Improve the error messages.

2. Package a third `Capability` as a jar in a `plugins/` directory and load it with `URLClassLoader` without compiling anything at run time.
3. Hold the version-1 loader only through a `WeakReference`, drop every other reference, call `System.gc()` and check whether it is collected. Then keep one plugin instance alive and repeat.
4. Replace `URLClassLoader` by your own `ClassLoader` subclass that reads class bytes from a `Map<String, byte[]>` and overrides `findClass`. (Stage 07 needs this.)

Takeaways

- Classes are loaded by loader objects in a parent-first chain: bootstrap, platform, application, and then whatever you create.
- Loading, linking and initialization are separate; the static initializer runs at first active use, and `Class.forName(name)` forces it.
- A class is identified by name *and* loader. Same bytes, different loader: different class.
- Configuration by name, `ServiceLoader` registration and custom loaders are the three ways code arrives late.
- A loaded class is executed code; trust is the whole question.

STAGE 06

Inversion of control

In which nobody writes `new` any more, a container builds the whole object graph from annotations, and we find out where control went.

Where we stand

By the end of stage 05 a class can be chosen by configuration, discovered by registration and loaded from anywhere. But somebody still has to *create the objects and connect them*. In every stage so far that somebody was `Main`, and in stage 06's own first section it still is:

```
Candidate candidate = new Candidate(1);
Composite byHand = new Composite(candidate, new DependencyB(), 10);
byHand.init();
```

Three objects, three lines. A real application has three hundred objects, and the lines must be in dependency order, and every time a constructor gains a parameter every `new` must be revisited. Worse, the alternative to wiring in `Main` is that objects create their own collaborators, which welds them together and makes them untestable.

Stage 06 builds the thing that takes this job away from `Main`: a dependency-injection container. It is about 150 lines, and every one of them is something from an earlier chapter.

Dependency injection by hand

Before the container, the principle, because the two are often confused. The component that needs collaborators looks like this:

```
@Component
public final class Composite {
    private final Capability capability;
    private final Dependency dependency;
    private final int limit;
    private String state = "constructed, not yet initialized";

    @Inject
    public Composite(@Named("candidate") Capability capability,
                     @Named("beta") Dependency dependency,
                     @Value("${composite.limit}") int limit) {
        this.capability = capability;
```

```
        this.dependency = dependency;
        this.limit = limit;
    }

    @PostConstruct
    void init() {
        state = "initialized with " + capability.getClass().getSimpleName()
                + ", " + dependency.name() + " and limit " + limit;
    }

    public String run() {
        capability.f(limit, 1.0);
        return "ran f(" + limit + ", 1.0) on " + capability;
    }
}
```

stage06-ioc/src/main/java/sa4e/reflect/Composite.java (abridged)

Ignore the annotations for a moment and look at the constructor. `Composite` does not create a `Capability` or a `Dependency`; it *receives* them. That alone is dependency injection. It is a design principle, not a framework feature, and it has an immediate benefit: `Composite` can be constructed in a test with a fake `Dependency` and no container anywhere in sight:

Expression	Result
`new Composite(new Candidate(1), new DependencyB(), 10); init()`	`initialized with Candidate, beta and limit 10`
`byHand.run()`	`ran f(10, 1.0) on Candidate{x=-10, label='candidate#1'}`

The container that follows automates the `new` calls. It does not change the principle.

The alternative, and why it hides things

There is an older pattern that solves the same problem differently, and it is worth seeing once so that you recognise it in legacy code:

```
static final class Locator {
    private static final Map<Class<?>, Object> REGISTRY = new HashMap<>();
    static <T> void put(Class<T> type, T instance) { REGISTRY.put(type, instance); }
    static <T> T get(Class<T> type) { /* look up or throw */ }
}

static final class LocatorUser {
    private final Dependency dependency = Locator.get(Dependency.class);   // pulled
    String describe() { return "using " + dependency.name(); }
}
```

Expression	Result
`new LocatorUser() before anything is registered`	`IllegalStateException: nothing registered for Dependency`
`Locator.put(Dependency.class, new DependencyA()); new LocatorUser().describe()`	`using alpha`

The *service locator* is a global registry that components pull from. It works, and it is why it is called an anti-pattern rather than a mistake: `LocatorUser`'s constructor takes nothing, yet `new LocatorUser()`

fails unless some global setup ran first. Its dependency is invisible in its signature, invisible to the compiler, invisible to the person writing a test. Constructor injection makes the dependency explicit, which is the whole argument for injection over location.

The container finds its components

Now the framework. Step one is knowing what exists:

```
Container container = new Container(config);
container.scan("sa4e.reflect");
```

`scan` uses a `ClassScanner` that does what stage 05's `ServiceLoader` does for one interface, for every class: it asks the loader for every location of the package (`getResources`), walks directories and jar entries, and calls the three-argument `Class.forName(name, false, loader)` on each name so that no static initializer runs. Of the result, the container keeps what carries `@Component`:

Name	Class	Scope
`alternative`	Alternative	singleton
`candidate`	Candidate	singleton
`chicken`	Chicken	singleton
`composite`	Composite	singleton
`alpha`	DependencyA	singleton
`beta`	DependencyB	singleton
`egg`	Egg	singleton
`ephemeral`	Ephemeral	prototype
`listener`	Listener	singleton
`ping`	Ping	singleton
`pong`	Pong	singleton

A component's name is the value of `@Component("alpha")` or, by default, the decapitalised simple class name. Nothing has been created yet. The JDK has no API for "list all classes in a package", so every container writes this scanner, and it is the reason a large Spring application spends its first seconds reading class files.

One request builds the graph

```
Composite composite = container.get(Composite.class);
```

Expression	Result
`container.get(Composite.class).state()`	`initialized with Candidate, beta and limit 10`
`composite.run()`	`ran f(10, 1.0) on Candidate{x=-10, label='candidate#0', dependency=alpha}`

One call, and `Composite` exists, fully wired. The container keeps a trace of what it did, and the trace is the best explanation of how a container works:

```
scanned sa4e.reflect: 11 components
construct Candidate with 0 argument(s)
construct DependencyA with 0 argument(s)
ready: DependencyA (singleton)
inject Candidate.dependency = the DependencyA instance
ready: Candidate (singleton)
construct DependencyB with 0 argument(s)
ready: DependencyB (singleton)
config Composite parameter limit = 10
construct Composite with 3 argument(s)
post-construct Composite.init()
ready: Composite (singleton)
```

Read it top to bottom. To build `Composite` the container looked at its `@Inject` constructor and found three parameters. The first, `@Named("candidate") Capability`, resolves to `Candidate`; building `Candidate` revealed an `@Inject` field of type `Dependency` named `alpha`, so `DependencyA` was built first and injected. The second parameter resolves to `DependencyB`. The third is a `@Value`, read from `application.properties` and converted to `int`. With all three in hand, `Composite` was constructed, its `@PostConstruct` method was called, and it was stored as a singleton. Depth-first, dependencies before dependents, and nobody wrote a single `new`.

The code behind the trace is short enough to read in full. Resolution is stage 01:

```
private Class<?> resolve(Class<?> type, String name) {
    List<String> matches = new ArrayList<>();
    for (Map.Entry<String, Class<?>> e : components.entrySet()) {
        boolean nameFits = name == null || name.equals(e.getKey());
        if (type.isAssignableFrom(e.getValue()) && nameFits) {
            matches.add(e.getKey());
        }
    }
    if (matches.isEmpty()) throw new ContainerException("no component for " + request);
    if (matches.size() > 1) throw new ContainerException("ambiguous request for " +
request
            + ": " + matches + "; use @Named");
    return components.get(matches.getFirst());
}
```

Construction is stage 02 with annotations from stage 03:

```
private Object construct(Class<?> implementation) {
    Constructor<?> constructor = chooseConstructor(implementation); // @Inject or no-arg
    Parameter[] parameters = constructor.getParameters();
    Object[] arguments = new Object[parameters.length];
    for (int i = 0; i < parameters.length; i++) {
        arguments[i] = valueFor(parameters[i].getType(), parameters[i], /* where */);
    }
    constructor.setAccessible(true);
    return constructor.newInstance(arguments);
}

/** What goes into a parameter or field: a configuration value, or another component. */
private Object valueFor(Class<?> type, AnnotatedElement element, String where) {
    Value value = element.getAnnotation(Value.class);
```

```
    if (value != null) {
        String key = value.value().replaceAll("^\\$\\{|}$", "");
        return convert(config.getProperty(key), type);
    }
    Named named = element.getAnnotation(Named.class);
    return instance(resolve(type, named == null ? null : named.value()));   // recursion
}
```

stage06-ioc/src/main/java/sa4e/reflect/Container.java (abridged)

`valueFor` is where the recursion lives: a parameter that is not a configuration value is another component, so `instance` is called for it, which calls `construct`, which calls `valueFor` for *its* parameters. The whole object graph is built by this recursion. Field injection and `@PostConstruct` are two more loops over `getDeclaredFields()` and `getDeclaredMethods()`, with `setAccessible(true)` and `invoke`.

Qualifiers

Two components implement `Capability`. Ask for it without saying which:

Expression	Result
`container.get(Capability.class)`	`ContainerException: ambiguous request for Capability: [alternative, candidate]; use @Named`
`container.get(Capability.class, "alternative")`	`Alternative{x=0}`
`container.get(Dependency.class, "alpha").name()`	`alpha`
`container.get(Runnable.class)`	`ContainerException: no component for Runnable`

The container refuses to guess, and says what it found. That is a deliberate choice and the right one: a container that silently picks the first match produces bugs that appear when a second implementation is added months later. `@Named` on the parameter or field settles it, and real containers add `@Primary`, profiles and conditions on top of the same loop.

Scopes

Expression	Result
`container.get(Candidate.class) == container.get(Candidate.class)`	`true`
`the Candidate inside Composite is that same instance`	`true`
`container.get(Ephemeral.class)`	`tmp#1`
`container.get(Ephemeral.class) again`	`tmp#2`
`container.get(Ephemeral.class) == container.get(Ephemeral.class)`	`false`

By default a component is a *singleton*: one instance per container, created on first request and shared by everyone who asks. `Composite` and `Main` got the same `Candidate`. A class marked `@Prototype` yields a new instance per request. The implementation is one `if` and one map; web frameworks add request and session scopes with the same shape and a proxy from stage 04 in front, so that a singleton can hold a reference to “the current request’s” object.

Field injection reaches into Candidate

`Candidate` changed for this stage, for the first time since stage 04:

```
@Component
public class Candidate implements Capability {
    @Inject
    @Named("alpha")
    private Dependency dependency;     // filled by the container after construction
    // ...
}
```

Expression	Result
`container.get(Candidate.class)`	`Candidate{x=-10, label='candidate#0', dependency=alpha}`
`Candidate.class.getDeclaredField("dependency").getModifiers()`	`2` (`private`)

The field is private and there is no setter; the container writes it with `setAccessible(true)` exactly as in stage 02. This is *field injection*, and it is convenient: no constructor to maintain. It is also the style most experienced people advise against, because the dependency is invisible in the constructor, the object is incomplete between construction and injection, and a test must either use the container or use reflection itself. Prefer constructor injection; keep field injection for tests and for the chapter that needs to show it.

Cycles

What if `Chicken` needs an `Egg` in its constructor and `Egg` needs a `Chicken`?

Expression	Result
`container.get(Chicken.class)`	`ContainerException: circular dependency: Chicken -> Egg -> Chicken`
`container.get(Ping.class).pong.ping == ping`	`true`

Neither can exist first, and the container says so, with the path. The detection is a stack of "classes under construction": if a class is requested while it is already on the stack, the request is circular. `Ping` and `Pong` have the same cycle through `@Inject` *fields*, and that works, because an object can be constructed, registered as a singleton, and only then have its fields filled; by the time `Pong` asks for `Ping`, `Ping` already exists. Spring behaves the same way, and its documentation still recommends breaking the cycle instead of relying on this.

The other face of inversion of control

Dependency injection is the most important form of inversion of control, but not the only one. Stage 06 ends with a second one, built from the same parts:

```
public final class EventBus {
    private record Subscription(Object target, Method method, Class<?> eventType) {}
    private final List<Subscription> subscriptions = new ArrayList<>();
```

```
    public void register(Object subscriber) {
        for (Method m : subscriber.getClass().getDeclaredMethods()) {
            if (!m.isAnnotationPresent(Subscribe.class)) continue;
            m.setAccessible(true);
            subscriptions.add(new Subscription(subscriber, m, m.getParameterTypes()[0]));
        }
    }

    public int post(Object event) {
        int delivered = 0;
        for (Subscription s : subscriptions) {
            if (!s.eventType().isInstance(event)) continue;   // stage 01
            s.method().invoke(s.target(), event);              // stage 02
            delivered++;
        }
        return delivered;
    }
}
```

stage06-ioc/src/main/java/sa4e/reflect/EventBus.java (abridged)

Expression	Result
`bus.subscriptions()`	`[Composite.on(Event), Listener.onAnything(Object)]`
`bus.post(new Event("started"))`	`2 subscriber(s)`
`bus.post("a plain string")`	`1 subscriber(s)`
`composite.received()`	`[started]`
`listener.seen()`	`[Event Event[name=started], String a plain string]`

A handler is found by annotation and matched by the type of its single parameter; `Listener.onAnything(Object)` therefore receives everything. The bus calls the handlers; the handlers never call the bus. Guava's `EventBus` and Spring's `@EventListener` are this with threading and error handling.

Where control went

Look back at the stages and ask, for each, who calls whom:

- **Stage 00.** `Main` created every object and called every method. Control was entirely in application code.
- **Stage 03.** The test runner called the test methods. The test class only declared.
- **Stage 04.** The proxy handlers decided what happens around every call. The real object did not know.
- **Stage 06.** The container creates the objects and wires them. The event bus decides who is called when. `Main` has become `container.get(Composite.class)`.

Application code has been reduced to *declaring what it needs and what it offers*. That is what makes Spring, Jakarta EE and every other application framework feel like magic, and it is reflection (stage 02), annotations (stage 03), proxies (stage 04) and class loading (stage 05), arranged. The stage 06 container does not use proxies, but the exercises ask you to combine them, and the moment you wrap every component in a `LoggingHandler` on creation you have Spring AOP.

In the wild

- Spring's `ApplicationContext`, Jakarta CDI, Guice and Dagger are this container with thousands of features: profiles, conditions, `@Primary`, lifecycle phases, scoped proxies, AOP integration, configuration binding, and eager validation at startup.
- JUnit (stage 03), servlet containers, message listeners, GUI event loops and Spring's `@EventListener` are inversion of control without dependency injection: the framework decides when your code runs.
- Dagger is the compile-time variant: the same annotations, but an annotation processor generates the wiring code, and no reflection happens at run time.
- `jakarta.inject` standardises exactly our vocabulary: `@Inject`, `@Named`, `@Singleton`, `Provider`.

The bill

- Startup: scanning and reflection take time. Large Spring applications spend seconds here, which is why Spring AOT and GraalVM native image move the work to build time.
- Indirection: which implementation is injected is decided by annotations and configuration, not by code one can follow with "go to definition".
- Errors move from compile time to startup time, or, with lazy creation as here, to the first request. Real containers create all singletons eagerly for exactly this reason.
- Field injection and field-injection cycles are convenient and both hide design problems.

Try it yourself

1. Add eager initialization: `container.start()` creates every singleton at once, so that a missing dependency fails at startup. What happens with `Chicken`?
2. Add `@Primary`, so that an ambiguous request picks the primary candidate instead of failing.
3. Let the container register every component that has `@Subscribe` methods with the event bus automatically, so that the last section needs no explicit `register` calls.
4. Combine stages: wrap every component that implements an interface in stage 04's `LoggingHandler` when it is created. Where in `Container` does that belong, and what breaks when a component is requested by its concrete class?

Takeaways

- Dependency injection is a principle: objects receive collaborators instead of creating them. The container only automates the `new` calls.
- A container is classpath scanning plus a recursive `resolve`, `construct`, `inject`, `post-construct` loop, built entirely from stages 01 to 03.
- Ambiguity is an error, not a guess. Singletons are shared, prototypes are fresh. Constructor cycles fail; field cycles resolve.
- Inversion of control means the framework calls you. Injection and event buses are two faces of the same idea.

STAGE 07

Generating classes at run time

In which we skip the compiler entirely, write a class one instruction at a time, and give `Candidate` *a subclass it never asked for.*

Where we stand

Two limits survived to the last stage. Dynamic proxies (stage 04) can only implement interfaces, so a class without an interface cannot be intercepted; that is why Spring asks for interfaces, and why it falls back to "CGLIB" when there are none. And run-time compilation (stage 05) needs source code and a compiler, which is slow and is not available on a plain JRE.

Frameworks need neither. They write class files directly. The JVM does not care where the bytes of a class come from, only that they are well-formed, and since Java 24 the JDK ships an official API for reading and writing them: `java.lang.classfile`. This chapter uses it to look at what the compiler produced, then to produce something of our own, and finally to do the one thing stage 04 could not.

Reading bytecode

Before writing a class file, read one. `Bytecode.parse` wraps `ClassFile.of().parse(bytes)` and lists what it finds:

Expression	Result
`Candidate.class size`	`2393 bytes`
`major version`	`69 (Java 25)`
`methods`	`[<init>(int)void : 14 instructions, <init>()void : 6 instructions, <init>(String)void : 13 instructions, getX()int : 3 instructions, f(int,double)void : 13 instructions, reset()void : 4 instructions, secret()String : 4 instructions, created()int : 2 instructions, toString()String : 15 instructions]`

Constructors are methods called `<init>`; the static counter's increment is why `Candidate(int)` has fourteen instructions. Here is `f`, source and bytecode side by side:

```
public void f(int y, double d) { if (d < 0) x = y; else x = -y; }
```

```
dload_2
dconst_0
dcmpg
ifge
aload_0
iload_1
putfield sa4e/reflect/Candidate.x : int
goto
aload_0
iload_1
ineg
putfield sa4e/reflect/Candidate.x : int
return
```

The JVM is a stack machine with numbered local variable slots. In an instance method slot 0 is `this`, and the parameters follow: `y` in slot 1, `d` in slots 2 and 3, because a `double` takes two. `dload_2` pushes `d`, `dconst_0` pushes `0.0`, `dcmpg` compares them, `ifge` jumps to the `else` branch if the result is not negative. Each branch loads `this` and `y`, optionally negates, and stores into the field `x` with `putfield`. That is all a method is: a list of instructions over a stack and some slots. Every framework that generates code works at this level, for twenty years with the ASM library and now with the JDK's own API.

Writing a class from nothing

`Generator.implementation` produces a complete `Capability` implementation without a line of Java source. In Java it would read:

```
@Comment("generated at run time")
public final class Generated implements Capability {
    private int x;
    public int getX() { return x; }
    public void f(int y, double d) { x = y; }
    public String toString() { return "Generated{x=" + x + "}"; }
}
```

and with the ClassFile API it reads:

```
ClassDesc self = ClassDesc.of("sa4e.reflect.Generated");
byte[] bytes = ClassFile.of().build(self, cb -> {
    cb.withFlags(ClassFile.ACC_PUBLIC | ClassFile.ACC_FINAL);
    cb.withSuperclass(CD_Object);
    cb.withInterfaceSymbols(ClassDesc.of("sa4e.reflect.Capability"));
    cb.with(RuntimeVisibleAnnotationsAttribute.of(Annotation.of(
            ClassDesc.of("sa4e.reflect.Comment"),
            AnnotationElement.ofString("value", "generated at run time"))));
    cb.withField("x", CD_int, ClassFile.ACC_PRIVATE);

    cb.withMethodBody(INIT_NAME, MTD_void, ClassFile.ACC_PUBLIC, code -> code
            .aload(0)
            .invokespecial(CD_Object, INIT_NAME, MTD_void)      // super();
            .return_());

    cb.withMethodBody("getX", MethodTypeDesc.of(CD_int), ClassFile.ACC_PUBLIC, code ->
code
            .aload(0)
```

```
            .getfield(self, "x", CD_int)                        // this.x
            .ireturn());

    MethodTypeDesc fType = MethodTypeDesc.of(CD_void, CD_int, CD_double);
    cb.withMethodBody("f", fType, ClassFile.ACC_PUBLIC, code -> code
            .aload(0)
            .iload(1)                                           // y
            .putfield(self, "x", CD_int)                        // this.x = y
            .return_());
    // toString: new StringBuilder, append, append, toString; see the source
});
```

stage07-codegen/src/main/java/sa4e/reflect/Generator.java (abridged)

A `ClassDesc` names a class, a `MethodTypeDesc` names a signature, and the `code` builder has one method per JVM instruction. The result is a `byte[]`. To make it a class, we need the one thing stage 05's exercise asked for, a loader that exposes `defineClass`:

```
public final class ByteArrayClassLoader extends ClassLoader {
    public ByteArrayClassLoader() {
        super("generated", ByteArrayClassLoader.class.getClassLoader());
    }

    public Class<?> define(String name, byte[] bytes) {
        return defineClass(name, bytes, 0, bytes.length);
    }
}
```

```
Class<?> generated = loader.define("sa4e.reflect.Generated", bytes);
Capability instance = (Capability) generated.getDeclaredConstructor().newInstance();
```

Expression	Result
`Generator.implementation(...)` produced	`649 bytes`
`ClassFile.of().verify(bytes)`	`no errors`
`generated.getName()`	`sa4e.reflect.Generated`
`generated.getInterfaces()`	`[interface sa4e.reflect.Capability]`
`instance.f(21, 0.0); instance.getX()`	`21`
`instance.toString()`	`Generated{x=21}`
`generated.getAnnotation(Comment.class).value()`	`generated at run time`

No source file, no `javac`, no temporary directory. Six hundred and forty-nine bytes went from `Generator` into `defineClass`, and out came a class that implements `Capability`, runs, and even carries the `@Comment` annotation of stage 03, written as a class-file attribute and read back by `getAnnotation` as if a compiler had put it there. `ClassFile.verify` runs the same structural checks the JVM would, so a mistake in the instruction sequence is reported before the class is defined rather than as a `VerifyError` at an unhelpful moment.

A proxy for a class

Now the limit of stage 04. `Generator.interceptingSubclass` writes the class that CGLIB would write:

```
public class Candidate$$Intercepted extends Candidate {
    public Candidate$$Intercepted(int x) { super(x); }
    public int getX() { Interceptor.before("getX"); return super.getX(); }
    public void f(int y, double d) { Interceptor.before("f"); super.f(y, d); }
}
```

where `Interceptor.before` is a static hook that records the call. The bytecode of the generated `getX`:

```
ldc getX
invokestatic sa4e/reflect/Interceptor.before(String)void
aload_0
invokespecial sa4e/reflect/Candidate.getX()int
ireturn
```

Push the string, call the hook, push `this`, call the superclass method with `invokespecial` (which is how `super.` is spelled in bytecode), return. And it behaves exactly as a hand-written subclass would:

Expression	Result
`intercepted.getSuperclass()`	`class sa4e.reflect.Candidate`
`c instanceof Candidate`	`true`
`c.f(3, -1.0); c.getX()`	`3`
`c (Candidate's own toString)`	`Candidate{x=3, label='candidate#7'}`
`Interceptor.CALLS`	`[f, getX]`

`c instanceof Candidate` is `true` this time, which is the whole difference from stage 04. The generated object *is* a `Candidate`, with `Candidate`'s fields, `Candidate`'s constructor having run, and `Candidate`'s `toString`. Code that asks for a `Candidate` rather than a `Capability` accepts it. This is what Spring does for a bean without an interface, what Hibernate does to make a lazily loaded entity look like the entity, and what Mockito does to mock a class.

The requirements follow from the mechanism. The class must not be `final`, the intercepted methods must not be `final` or `private` (a subclass cannot override them), and the superclass constructor runs, with whatever side effects it has; `Candidate.created()` went up by one. Every one of these appears in the documentation of every framework that generates subclasses, usually as a rule you are asked to follow without being told why.

Hidden classes

Stage 01 ended with a lambda whose class was "hidden". Here is how such a class is made:

```
byte[] bytes = Generator.implementation("sa4e.reflect.Hidden");
Class<?> hidden = MethodHandles.lookup().defineHiddenClass(bytes, true).lookupClass();
```

Expression	Result
`hidden.getName()`	`sa4e.reflect.Hidden/0x00001e000104bc00`
`hidden.isHidden()`	`true`
`h.getX()`	`0`
`Class.forName(hidden.getName())`	`ClassNotFoundException`

A hidden class is defined through a `Lookup` rather than a loader. It has a name, but the name has a `/0x...` suffix that no other class can refer to and `Class.forName` cannot resolve; it lives in the same package as the lookup class; and it can be unloaded when its last instance is gone, independently of its loader. The JDK uses hidden classes for lambdas, for the proxies of stage 04 since Java 16, and for the glue classes behind `MethodHandle`. For a framework that generates thousands of small classes, "unloaded with the last instance" is the property that matters.

Looking at the result with javap

Generated classes are ordinary class files, and the stage writes them to disk so that the standard tools can inspect them:

```
javap -c -p /tmp/reflect-generated/Generated.class
```

Anything that reads class files, `javap`, a decompiler, an IDE, or `Bytecode.parse` from the first section, treats them like any other. There is no mark that says "generated"; the name `Candidate$$Intercepted` is a convention, chosen so that it is visible in stack traces, and the double dollar is a tradition from CGLIB (`$$EnhancerByCGLIB$$`) that Spring, Hibernate and ByteBuddy kept.

The whole spectrum

Step back from this chapter and look at all the ways code can be made to exist:

- **Compile time.** Annotation processors (Lombok, MapStruct, Dagger, the JPA metamodel) run inside `javac`, read `SOURCE`-retention annotations and generate source or class files. No reflection at run time, but a build step and a compiler plugin.
- **Run time, from source.** Stage 05, JSP engines and the Java shell call the compiler. Flexible, slow, needs a JDK.
- **Run time, from bytes.** This stage, `java.lang.reflect.Proxy`, lambdas, CGLIB, ByteBuddy and Mockito. Fast, needs no compiler, needs someone to write bytecode.
- **Ahead of time.** GraalVM native image resolves everything at build time and needs a complete list of what will be loaded, reflected on or proxied; dynamic generation at run time is simply not possible there.

The trend in Spring, Quarkus and Micronaut is to move from the third option towards the first and the fourth, trading run-time reflection for build-time generation, for faster startup and smaller memory. The concepts do not change; only the moment at which the decisions are made moves, which is where this book started.

> **In the wild**
>
> - `java.lang.reflect.Proxy` (stage 04) and `LambdaMetafactory` (stage 01) generate classes like the second section, inside the JDK itself.

- CGLIB, ByteBuddy and Javassist generate subclasses like the third section for Spring class proxies, Hibernate lazy entities, Mockito mocks and Jakarta CDI scoped beans.
- Before Java 24 all of them used the ASM library; the JDK itself has been moving to the ClassFile API, which was in fact built to replace ASM inside the JDK.
- Instrumentation agents (`java.lang.instrument`) use the same API to rewrite classes *as they are loaded*, which is how profilers and tracing agents work.

The bill

- Writing bytecode by hand is error-prone; the verifier catches structural mistakes, not logical ones. Frameworks keep this code small and test it exhaustively.
- Generated classes complicate debugging and stack traces (`Candidate$$Intercepted`, `$$EnhancerBySpringCGLIB$$`) and are invisible to static analysis.
- Each generated class costs metaspace; hidden classes help by dying with their last instance.
- None of it works in a closed-world ahead-of-time build.

Try it yourself

1. Extend `Generator.interceptingSubclass` so that the generated `getX` also reports the return value to `Interceptor`. (Hint: `dup` before the `invokestatic`.)
2. Generate the subclass for any class and any list of public non-final methods, with the list taken from reflection, and compare with stage 04's `LoggingHandler`.
3. Measure a call through the generated subclass against a stage 04 proxy and against a direct call.
4. Remove the `dload(2)` from `f` or swap two instructions on purpose and read what `ClassFile.verify` and then the JVM report.

Takeaways

- A class is bytes; the JVM accepts them from a compiler, from a generator or from a `Lookup`, as long as they verify.
- The ClassFile API reads and writes class files; a method is a list of instructions over a stack and numbered slots.
- A generated subclass is the class proxy that interfaces-only `Proxy` cannot provide; it requires non-final classes and methods.
- Hidden classes are unnamed, unreferenceable and unloaded with their last instance; lambdas are hidden classes.
- Code generation at compile time, at run time from source, at run time from bytes, and ahead of time are the same decisions made at different moments.

Conclusion

In which the chill is gone, and we count what we built.

What we built

Eight stages ago, `Main` created two `Candidate`s and printed them, and there were four things it could not do. Here is where those four ended up.

Describe a class without reading its source. Stage 01 asked a `Class` object about itself; stage 02′s `Inspector` listed every constructor, field, method and parameter of any class, and stage 03 taught it to list annotations it had never heard of. In stage 07 the same idea went one level down, to the instructions inside the methods.

Reach private members. Stage 02 did, with `setAccessible(true)`, and found the one wall the JVM puts in the way: its own modules, since Java 16. Every later stage used that door, the validator, the test runner, the container, without a second thought, which is exactly how frameworks use it.

Create an instance of a class named at run time. Stage 01 opened the door with `Class.forName`; stage 05 walked through it with configuration, `ServiceLoader`, a compiler called from inside the program and a loader of our own, and discovered that a class is identified by its name and its loader together.

Decorate, replace or wire without editing `Main`. Stage 04 wrapped `Candidate` in logging, validation, transactions and a network stub without touching it, using classes generated by `Proxy`; stage 06 took the wiring away from `Main` altogether, with a container that reads annotations and builds the object graph; stage 07 generated the subclass that `Proxy` could not.

Along the way we wrote, in a few hundred lines in total, a recognisable fraction of Bean Validation, JUnit, Spring AOP, RMI, JPA's lazy loading, `ServiceLoader`, a Spring-style `ApplicationContext`, Guava's `EventBus` and CGLIB. None of it is production quality, and all of it is the real mechanism.

The one idea

If the book has a single thesis, it is the one from stage 00: in plain Java every decision about which class, which method and which field is made by the compiler from source text, and meta-programming is the art of making some of those decisions *later*, with information that only exists later.

Reflection makes the decision at run time, from names. Annotations let the code being decided about have a say. Proxies and generated classes make the decision for code that does not exist yet. Class loaders decide where the code comes from at all. Inversion of control is what it looks like when a

framework makes all of these decisions and the application only declares. Each later stage is the same move, applied to a different decision.

The same thesis explains the current direction of the industry. GraalVM native image, Spring AOT, Quarkus and Micronaut are moving decisions *back* towards build time, because startup time and memory have become expensive in containers and serverless functions. They do not abandon the mechanisms of this book; they run them earlier and record the results. The annotations are the same, the proxies are generated at build time, the container's wiring is written out as code. Understanding the run-time version is the only way to understand what the build-time version is doing on your behalf.

What it costs, in one place

Every chapter had a bill, and the bills rhyme:

- **Names as strings.** No compiler check, no refactoring support, errors at run time or at startup instead of at compile time.
- **Speed.** A reflective call is an order of magnitude slower than a direct one. Reflect once, cache, call through something cheaper.
- **Encapsulation.** Reflection defeats it; the JDK and your own modules can push back.
- **Visibility.** Behaviour happens in handlers, containers and generated classes that are written nowhere near the code they affect. "Go to definition" stops working; stack traces grow.
- **Trust.** A loaded class is executed code, and Log4Shell is what happens when that is forgotten.
- **Closed worlds.** Everything decided at run time must be declared in advance for ahead-of-time compilation.

None of these is a reason not to use frameworks. All of them are reasons to know what a framework is doing, which was the point.

What to do next

Run the stages. Change them. The exercises at the end of each chapter are small on purpose: adding `@Primary` to the container or a `CachingHandler` to the proxy chain takes an hour and teaches more than reading the chapter twice.

Then open a real framework. Find `AopUtils.getTargetClass` in Spring and recognise `Proxies.unwrap`. Find `ServiceLoader` in a JDBC driver's jar. Put a breakpoint in a `@Transactional` method and read the stack trace from the bottom up: `$Proxy` or `$$SpringCGLIB$$`, an interceptor chain, a transaction manager, your code. You wrote each of those frames once. The chill should be gone.